Priority Queues: Binary Heaps

Definition

Binary max-heap is a binary tree (each node has zero, one, or two children) where the value of each node is at least the values of its children.

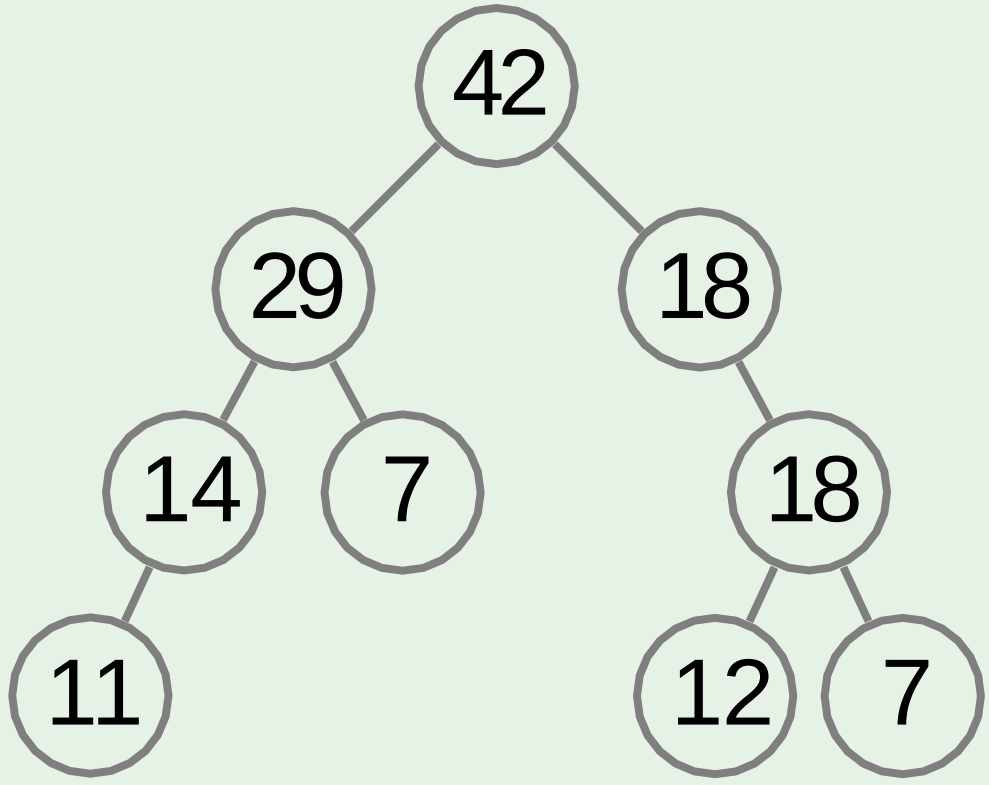
Definition

Binary max-heap is a binary tree (each node has zero, one, or two children) where the value of each node is at least the values of its children.

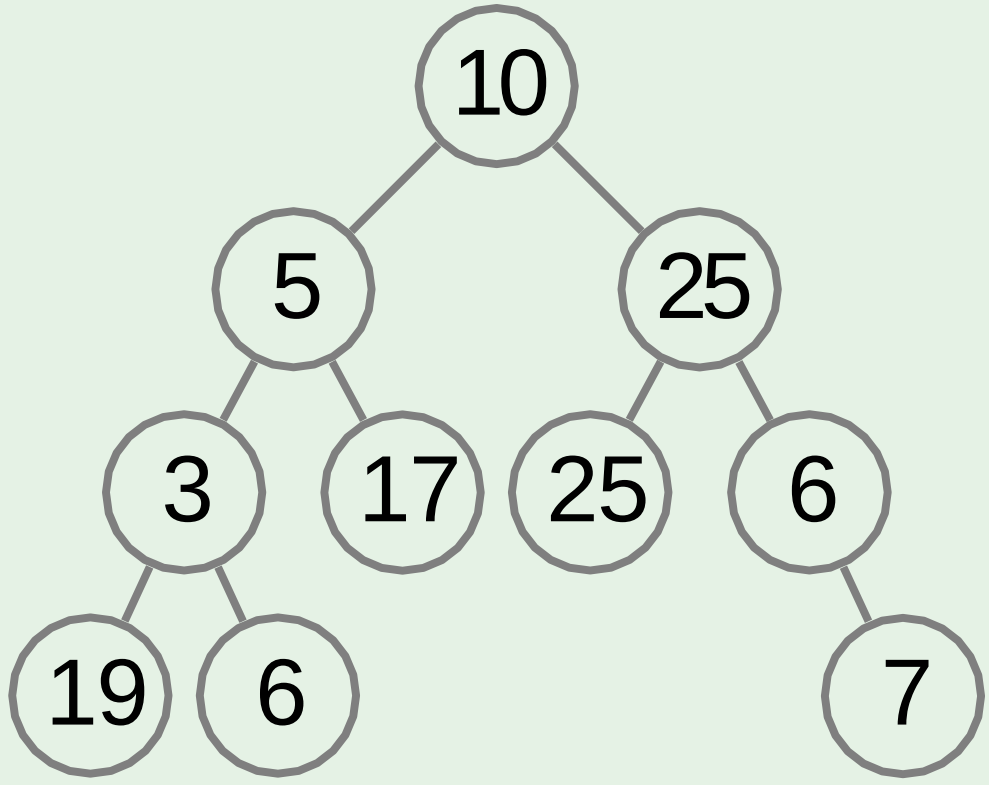
In other words

For each edge of the tree, the value of the parent is at least the value of the child.

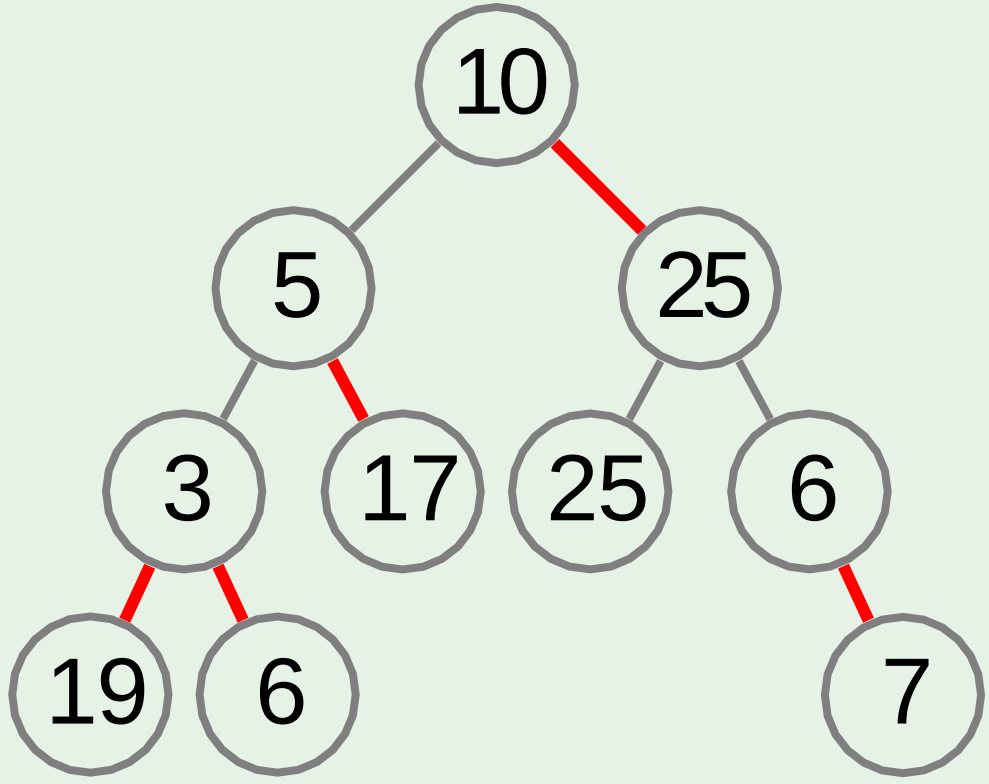
Example: heap



Example: not a heap

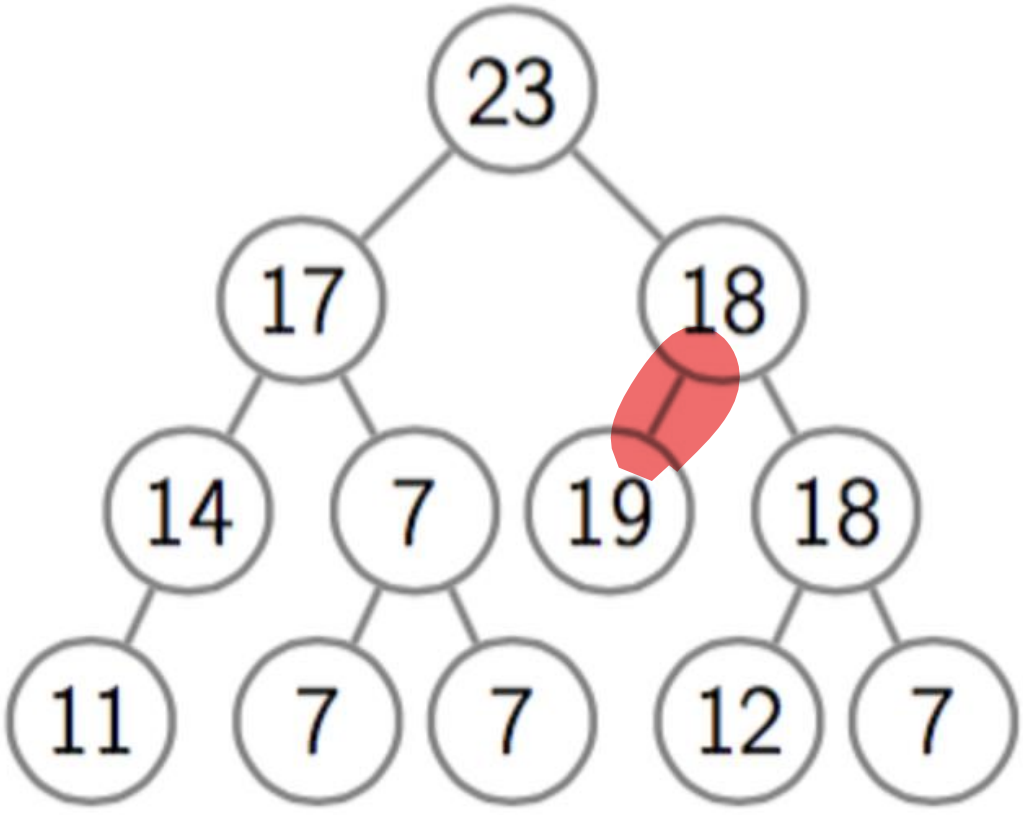


Example: not a heap

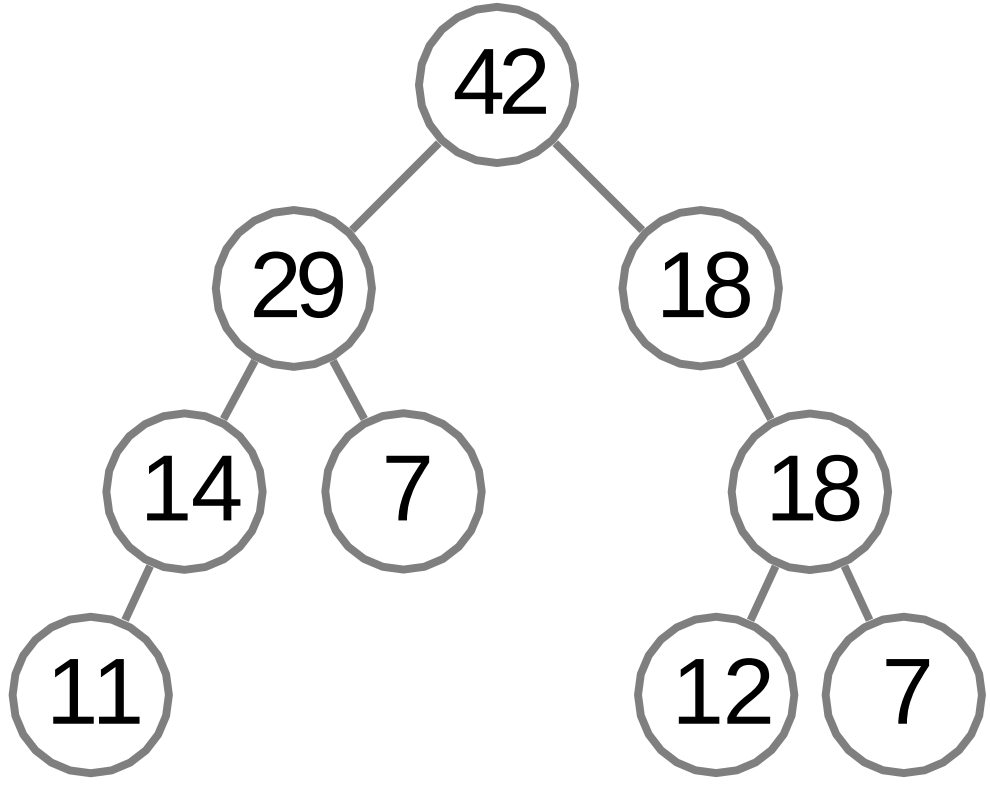


Question

Is it a binary max-heap? NO

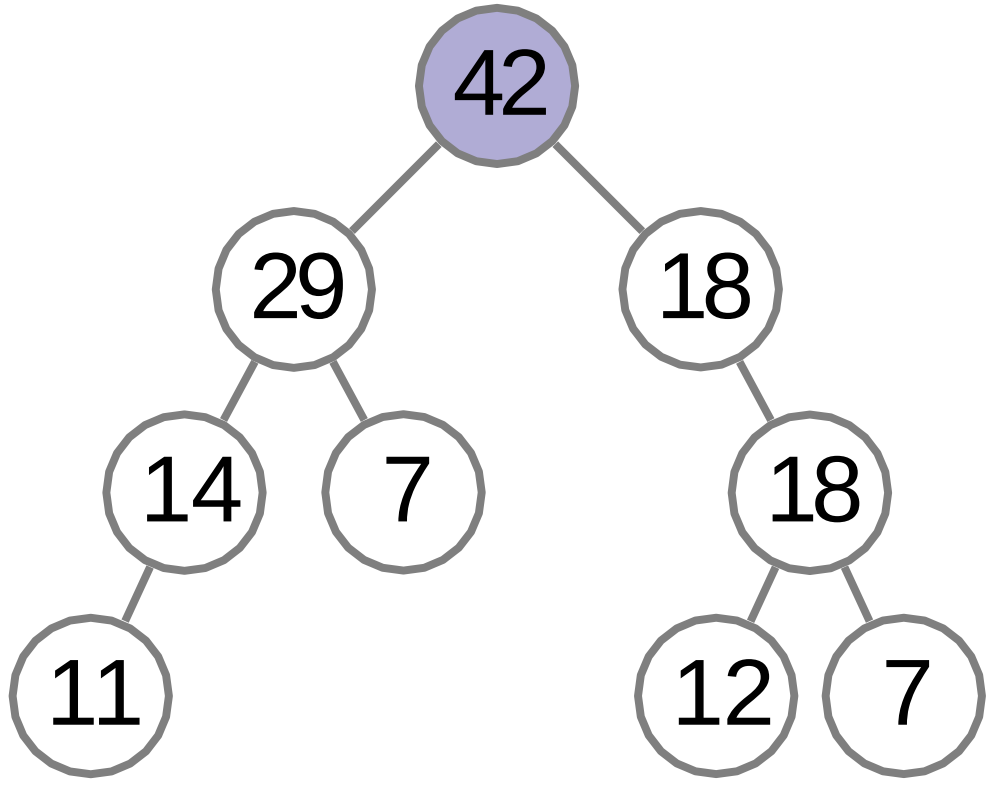


GetMax



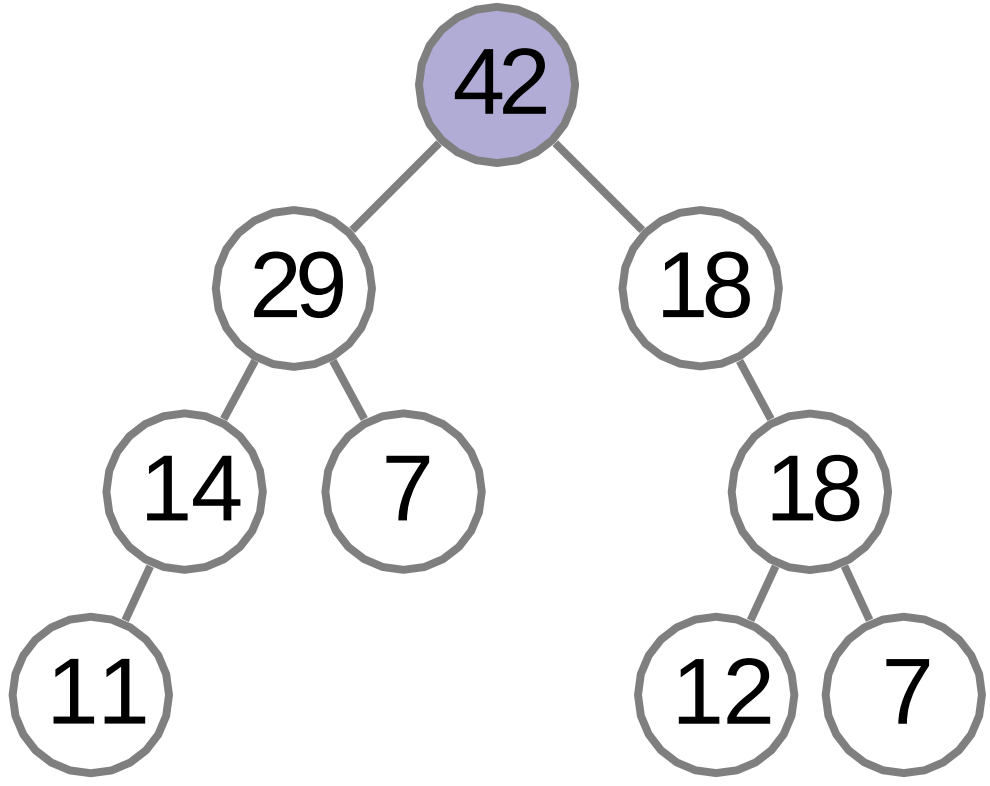
GetMax

return the root value



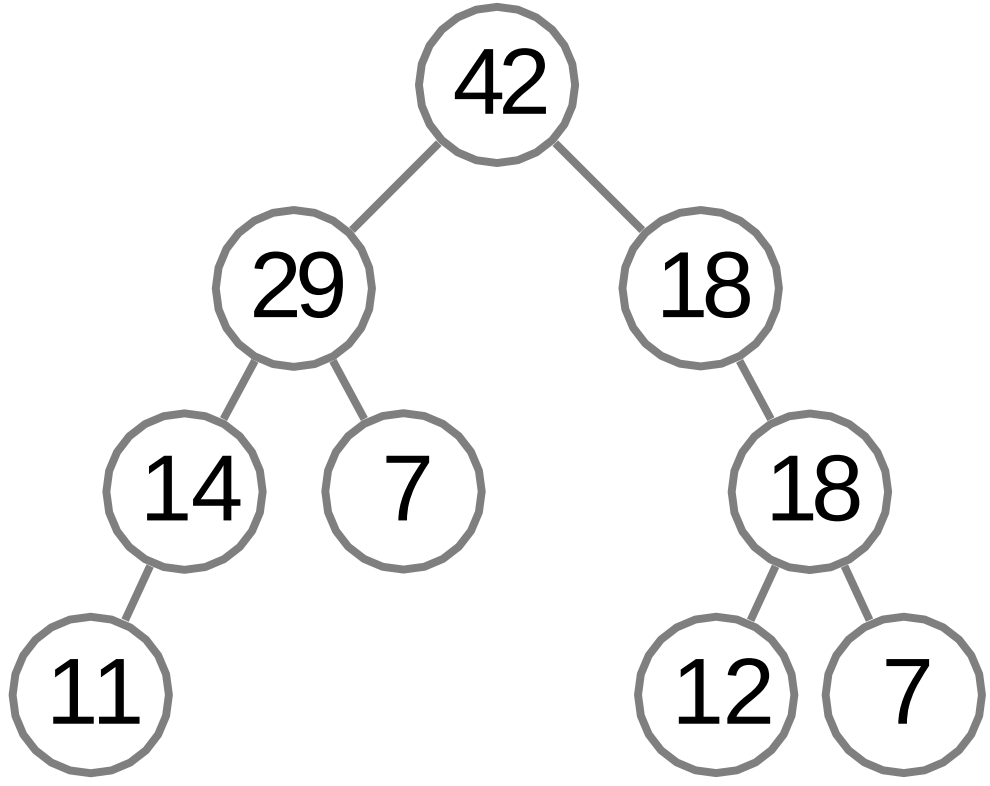
GetMax

return the root value



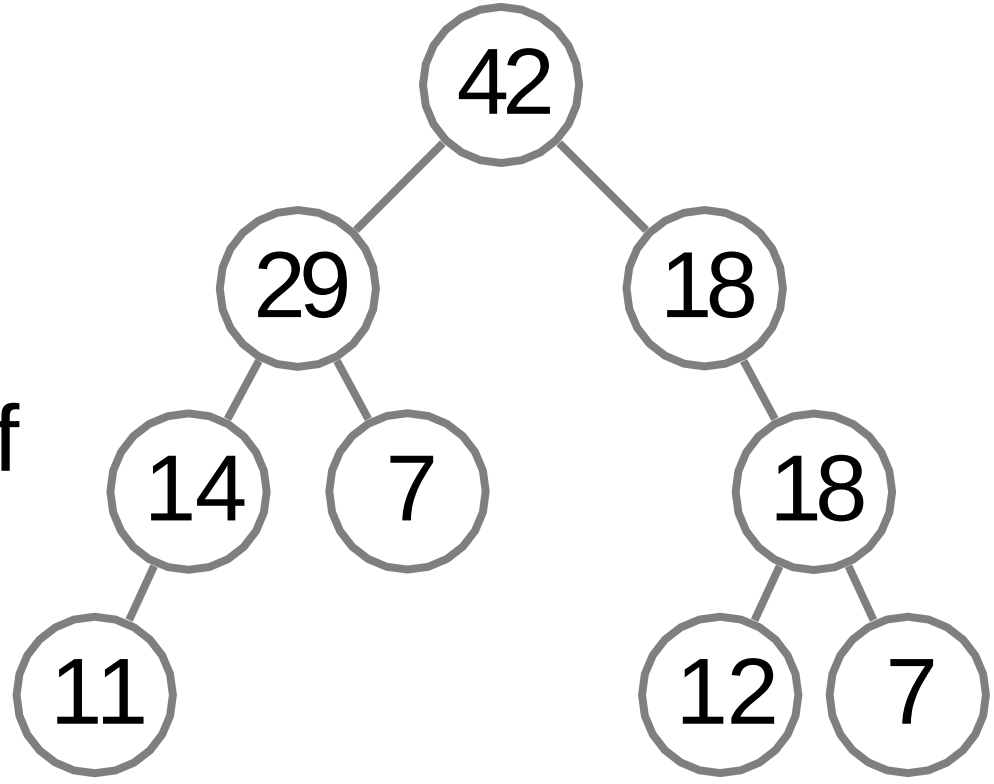
running time: $O(1)$

Insert



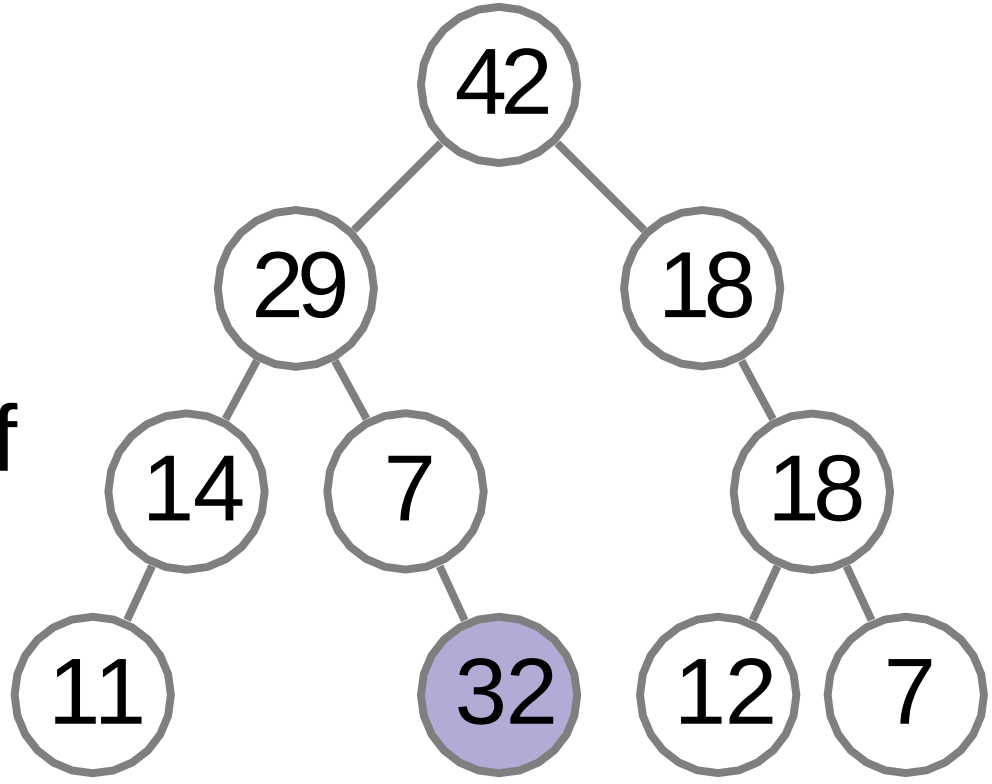
Insert

attach a new node to any leaf



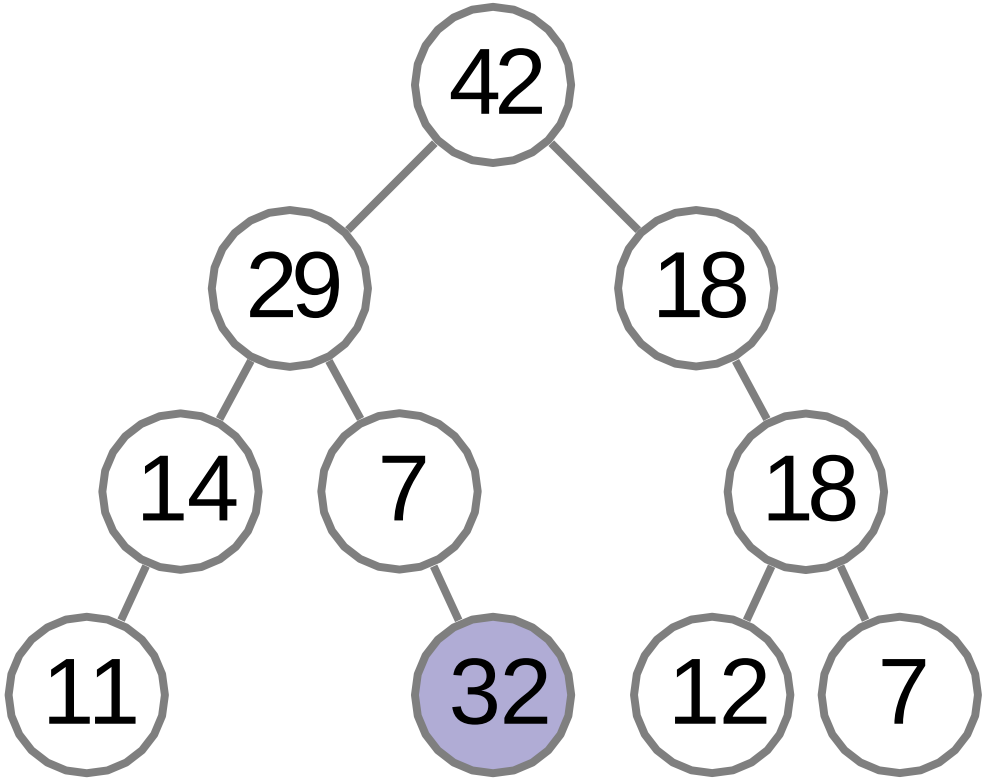
Insert

attach a new node to any leaf



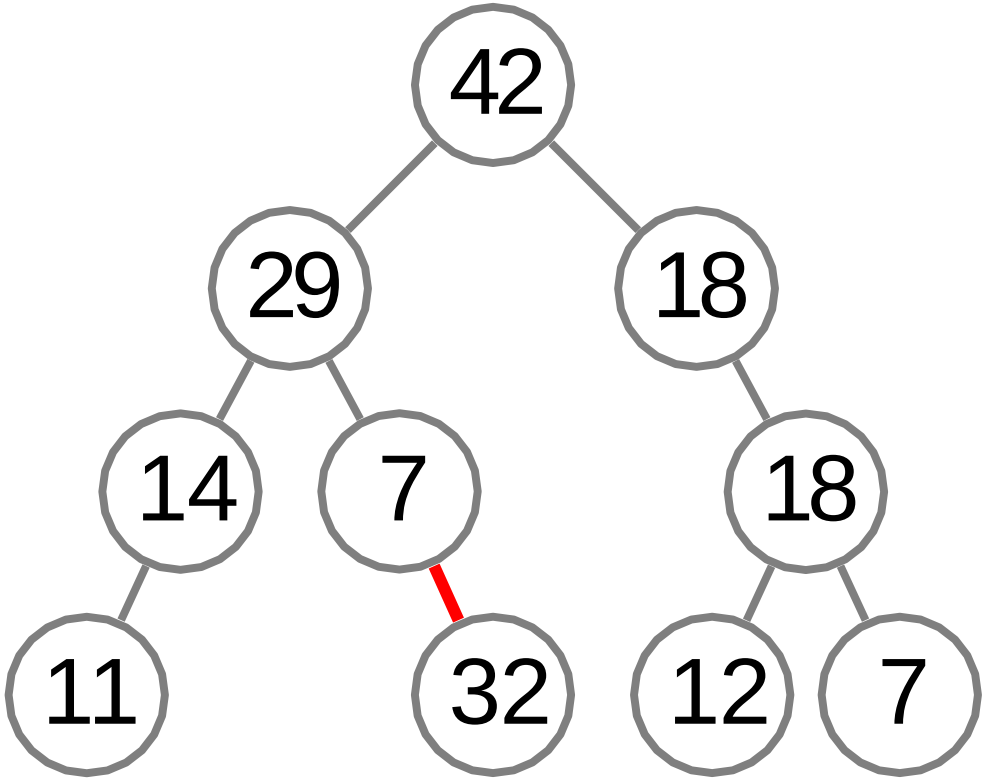
Insert

this may violate the heap property



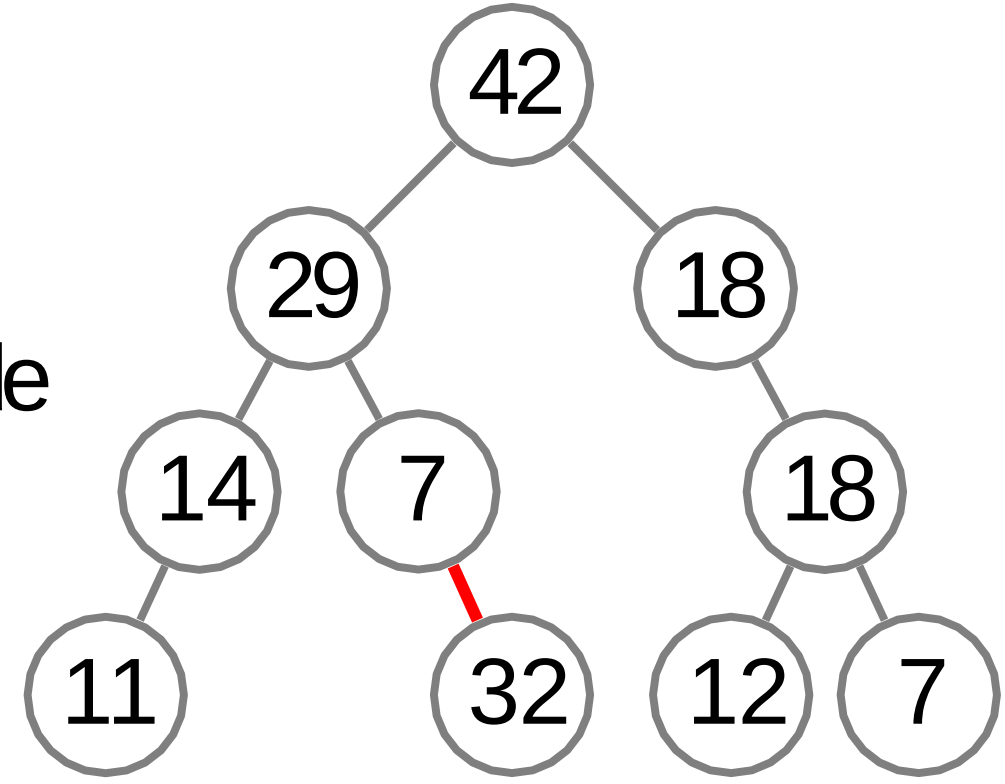
Insert

this may violate the heap property



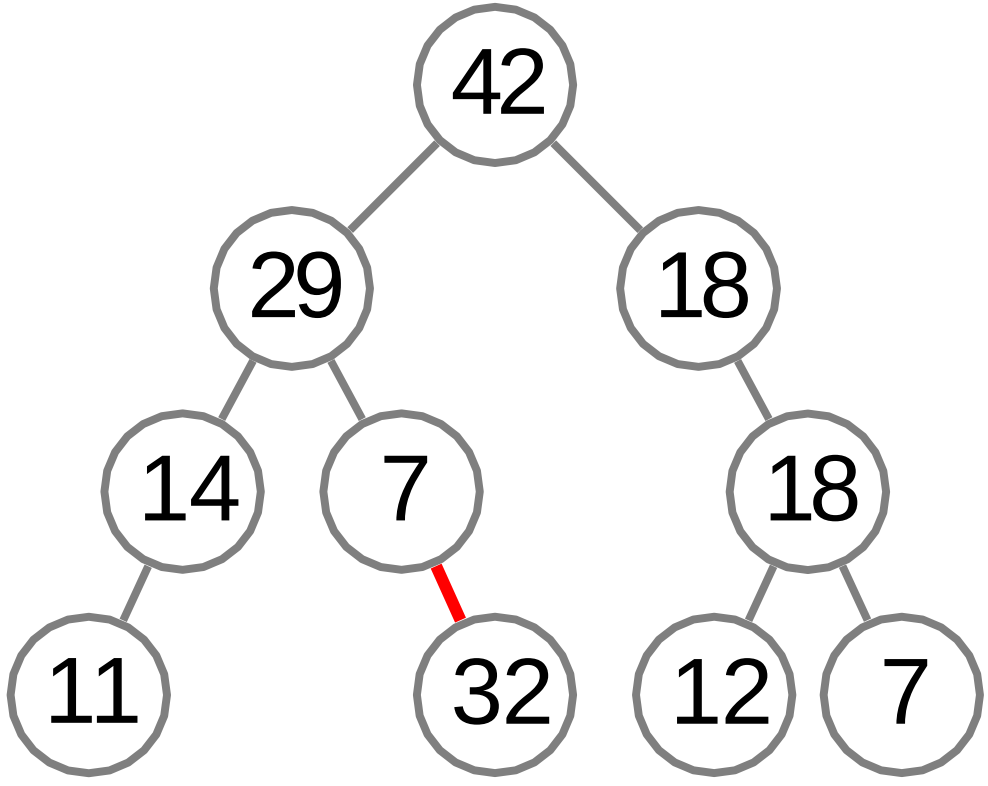
Insert

to fix this, we let the new node sift up

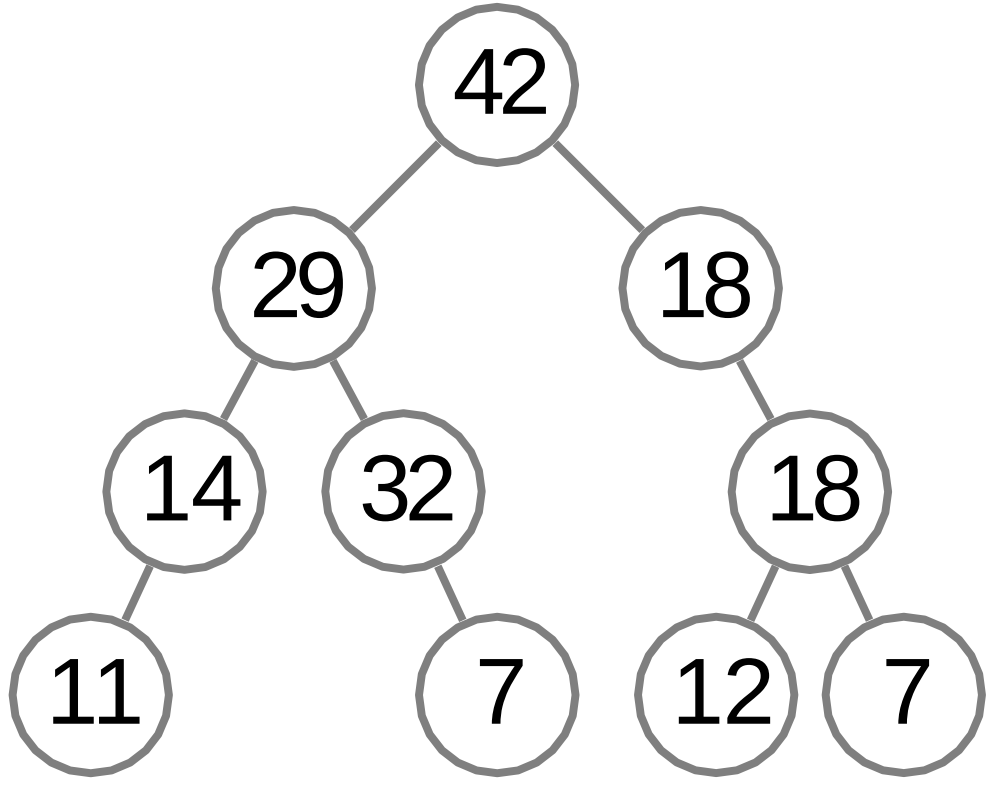


SiftUp

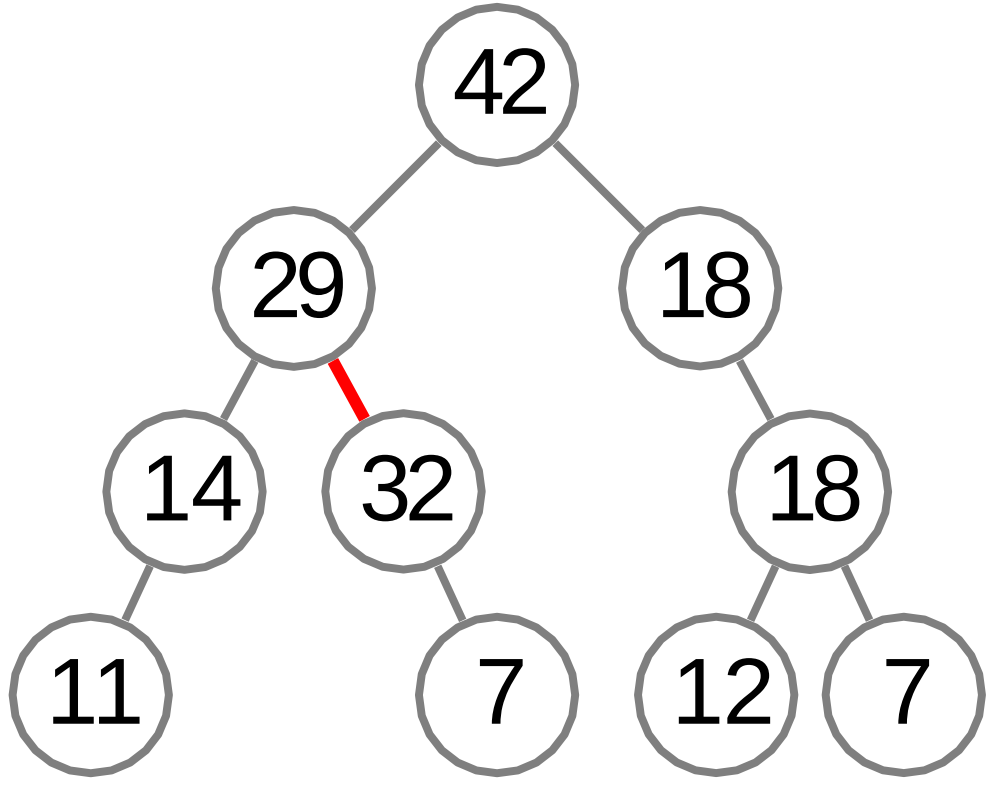
for this, we swap the problematic node with its parent until the property is satisfied



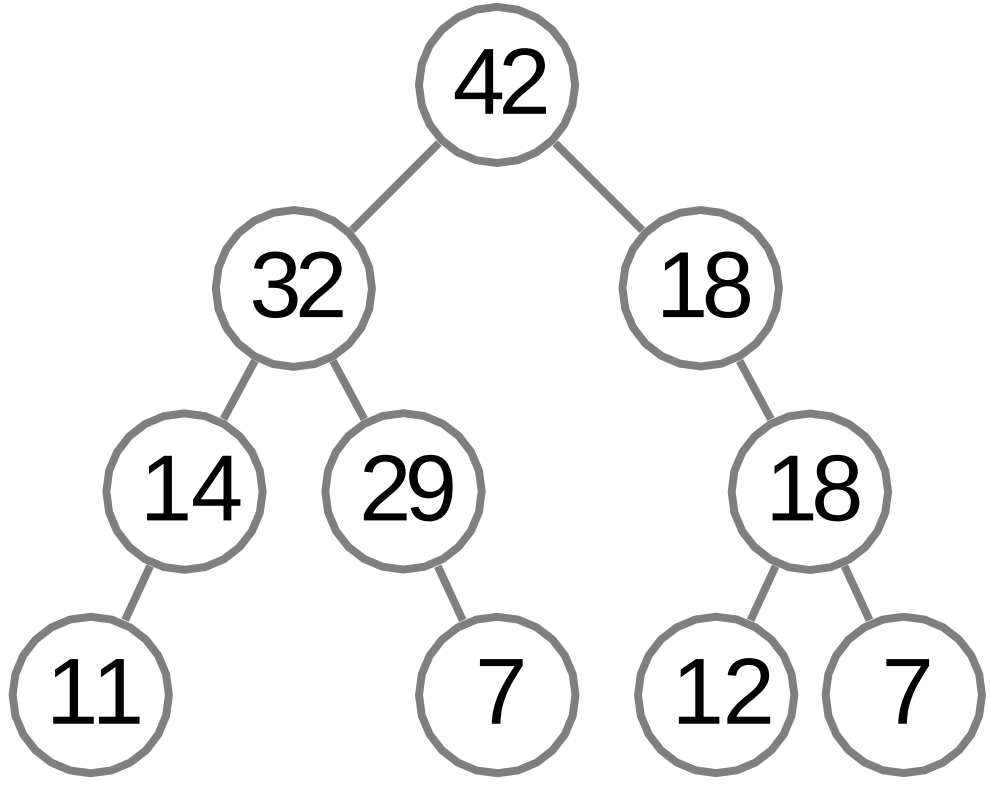
SiftUp



SiftUp

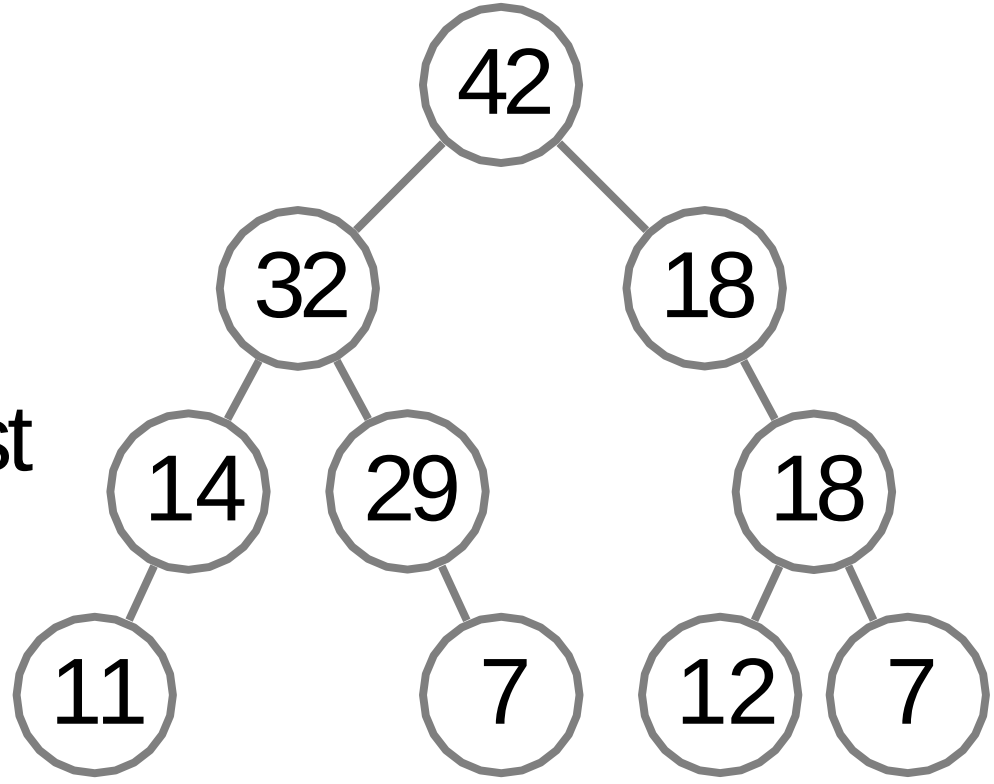


SiftUp



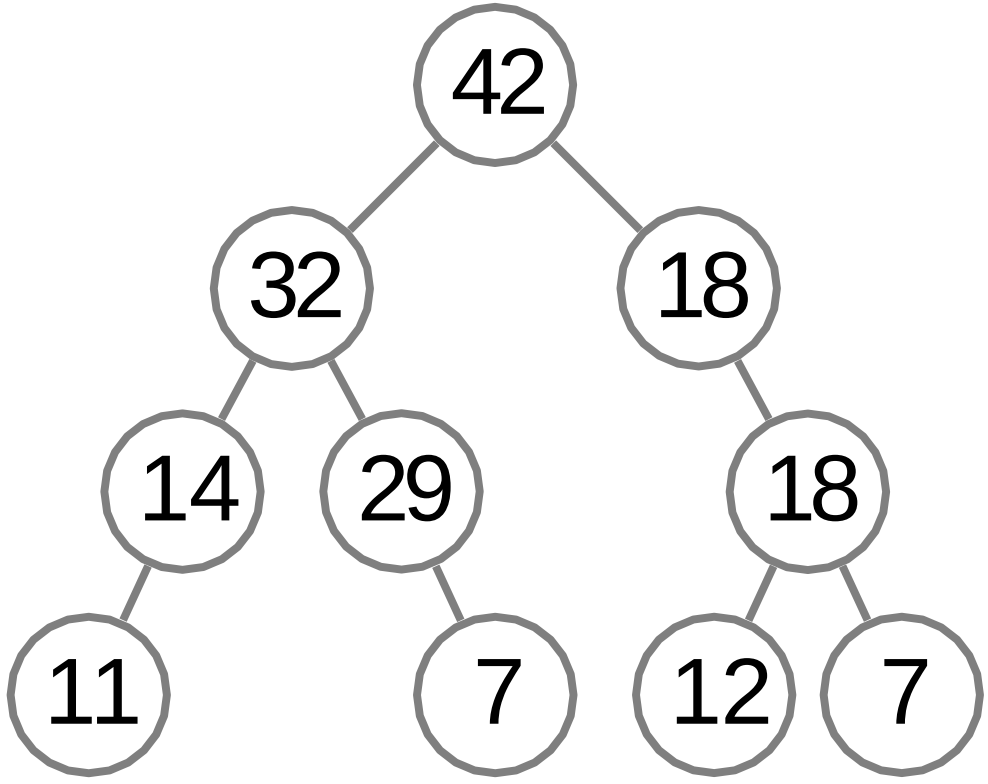
SiftUp

invariant: heap property is violated on at most one edge

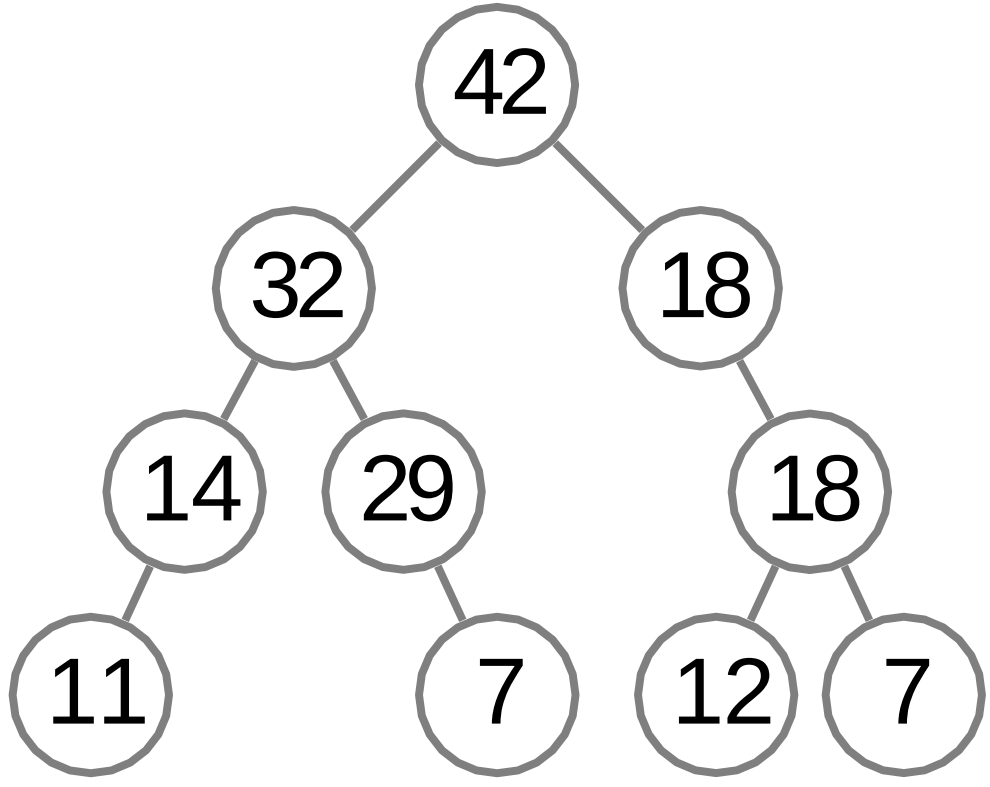


SiftUp

this edge gets closer to the root while sifting up



SiftUp



running time: *O*(tree height)

Question

Assume that initially a binary max-heap H contains just a single node with value 0. Then, for all i from 1 to n, we insert i into H. We do this by attaching a new element to some leaf of the current tree. What will be the height of the resulting tree?

$\Theta(n^2)$

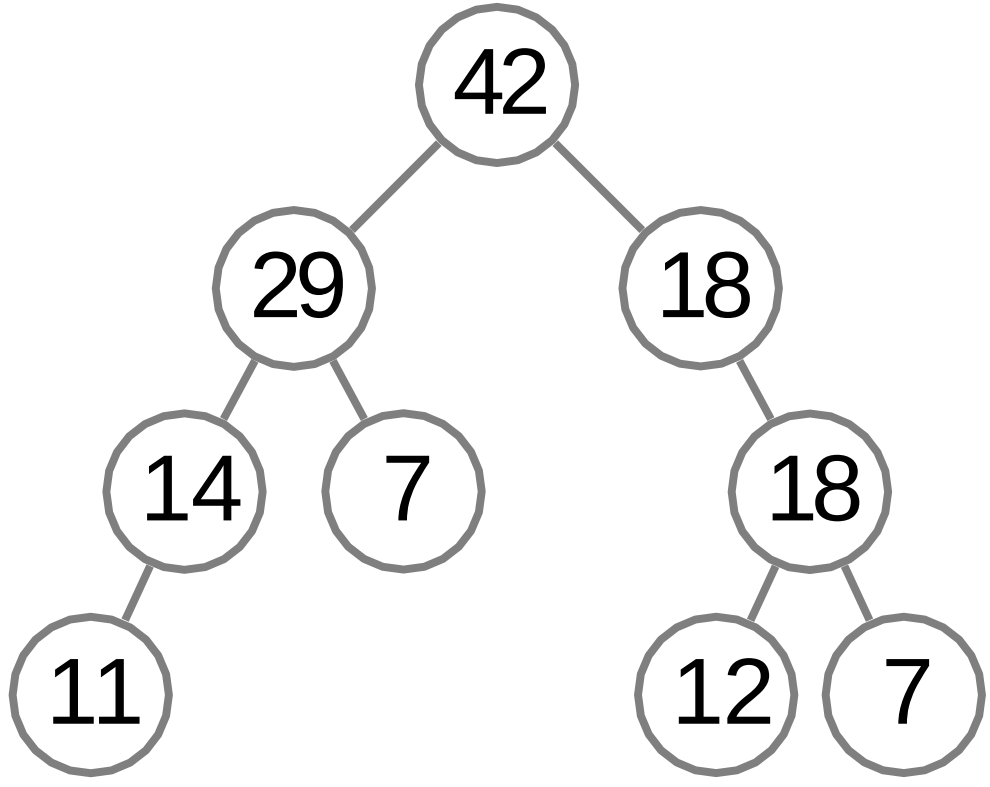
since the height of the tree is O(logn)

$\Theta(n)$

O(logn), the maximum number of swaps needed to restore the heap property during each insertion is proportional to the height.

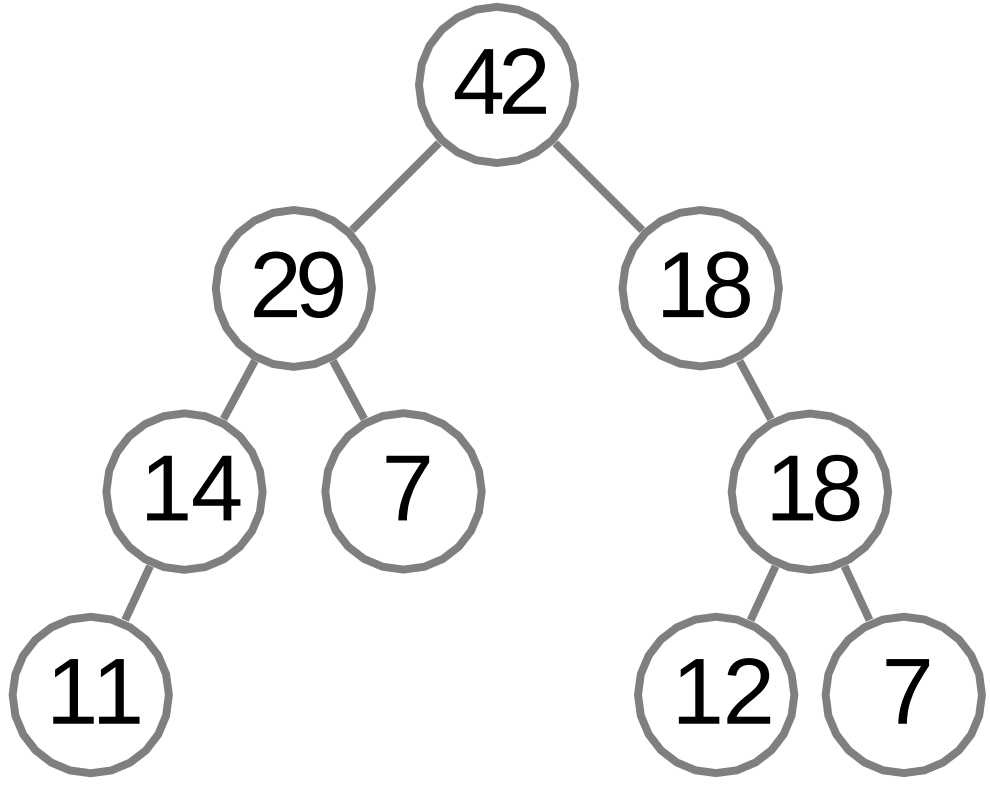
$\Theta(\log n)$

ExtractMax



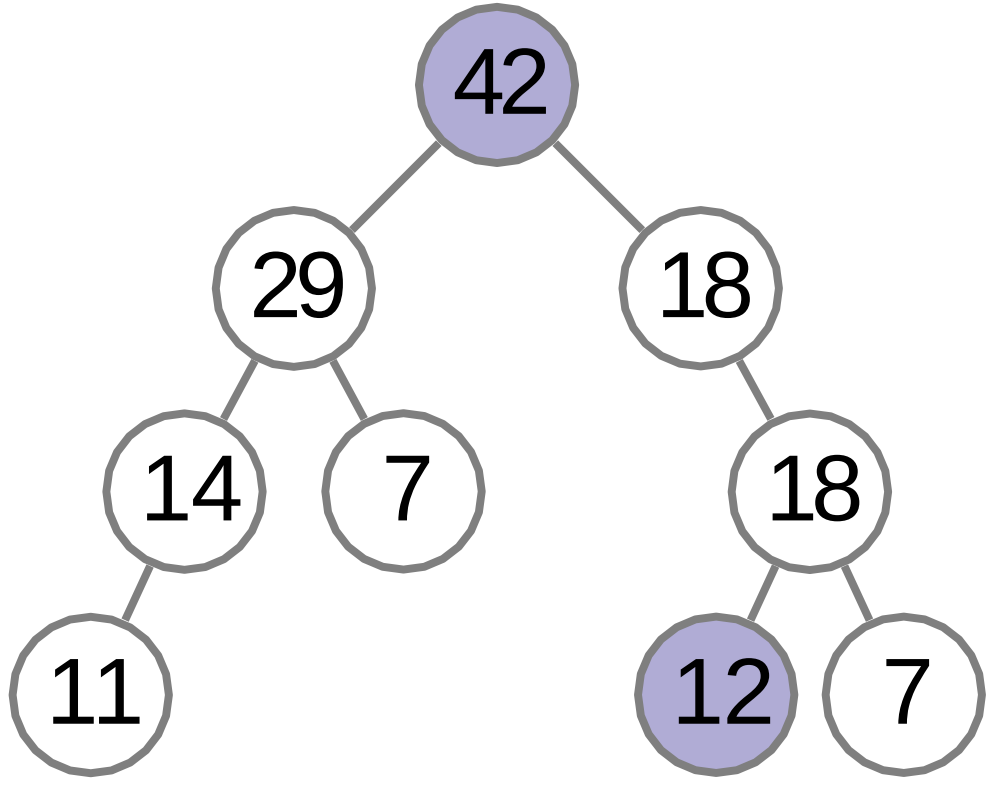
ExtractMax

replace the root
with any leaf



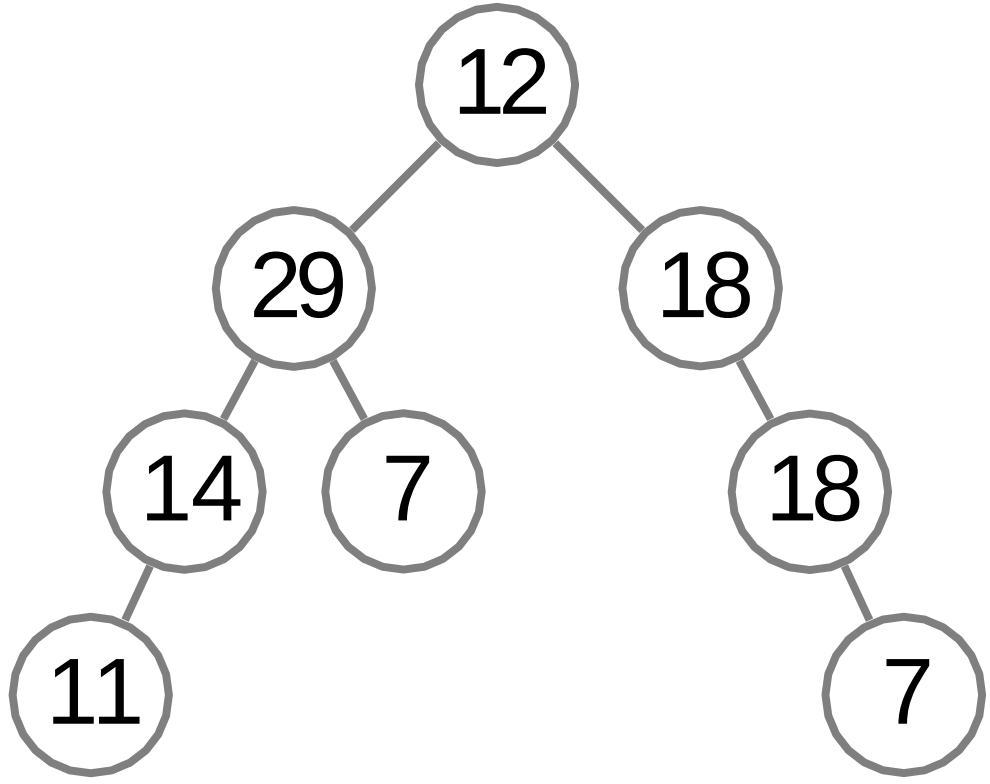
ExtractMax

replace the root
with any leaf



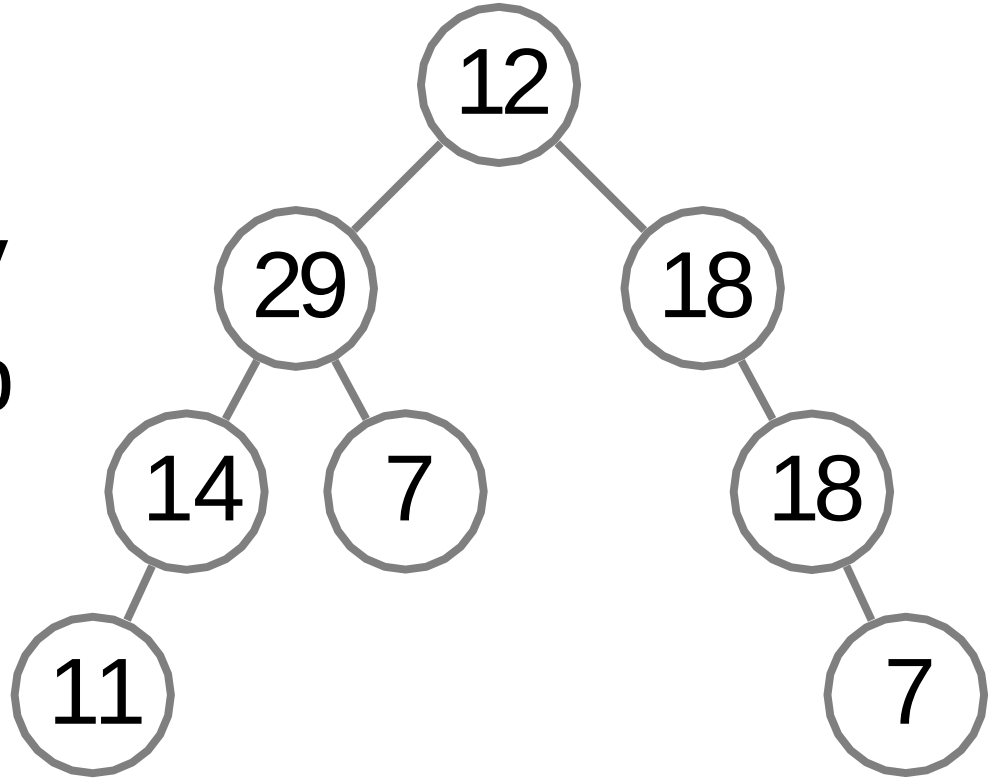
ExtractMax

replace the root
with any leaf



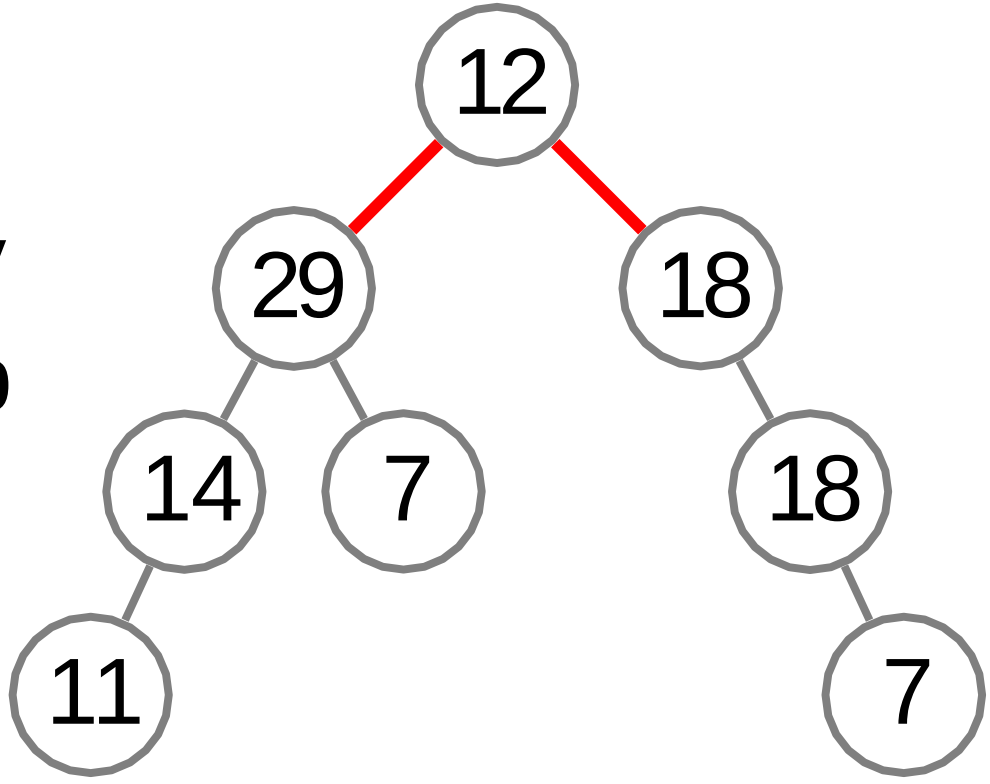
ExtractMax

again, this may violate the heap property



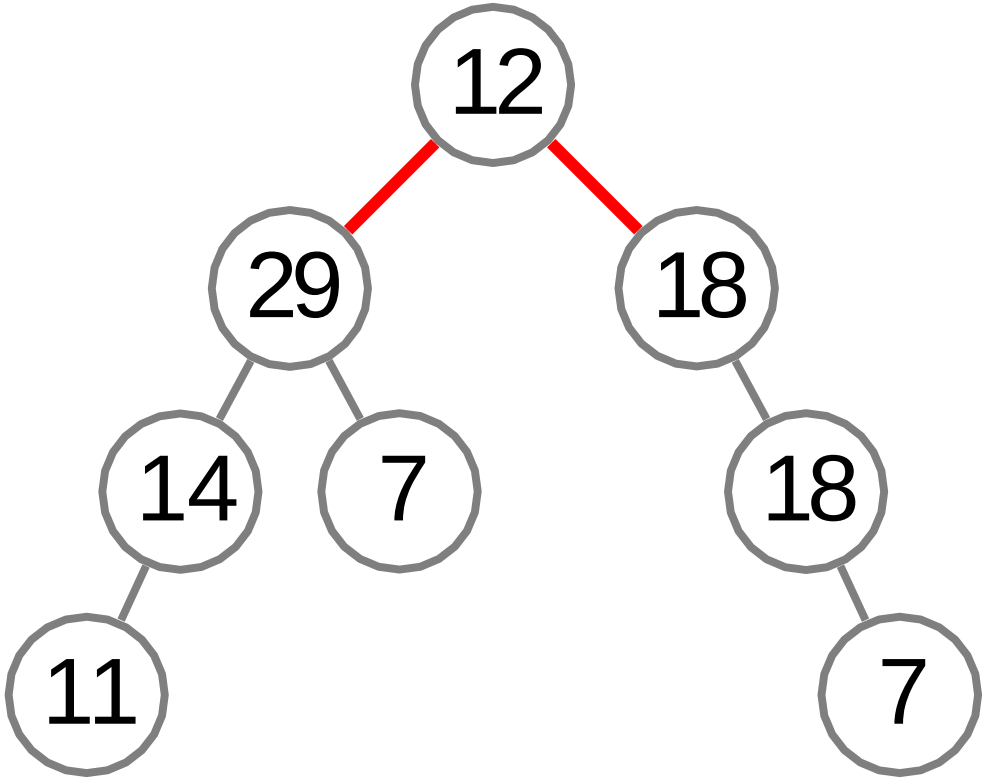
ExtractMax

again, this may violate the heap property



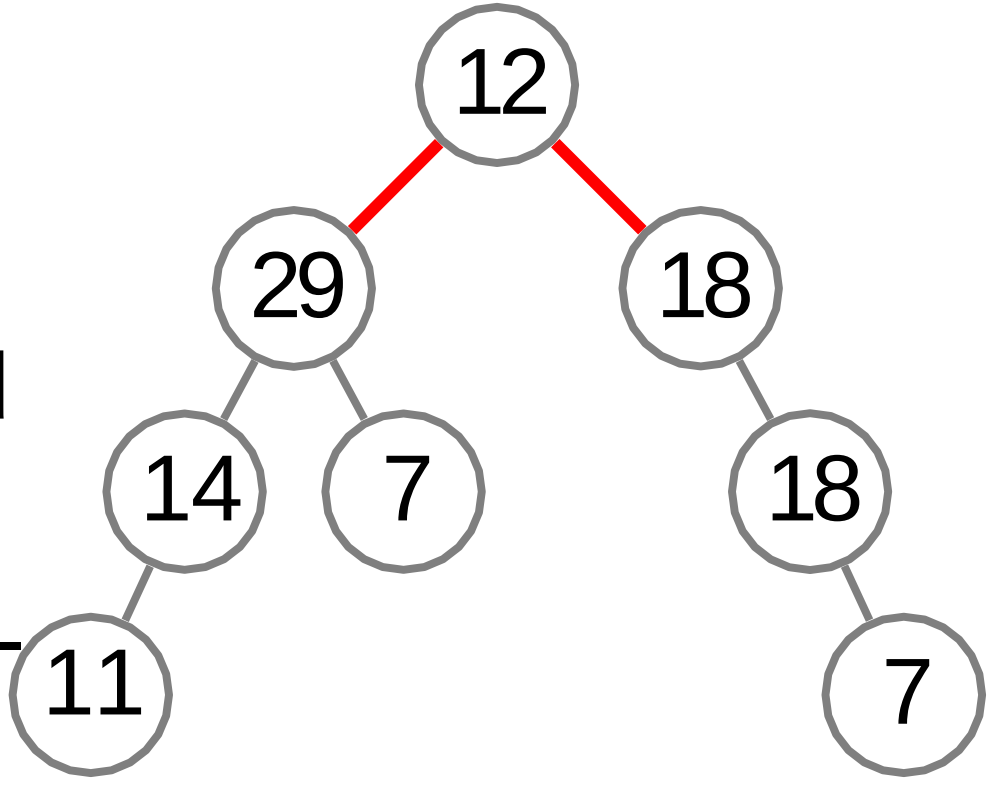
ExtractMax

to fix it, we let the problematic node sift down

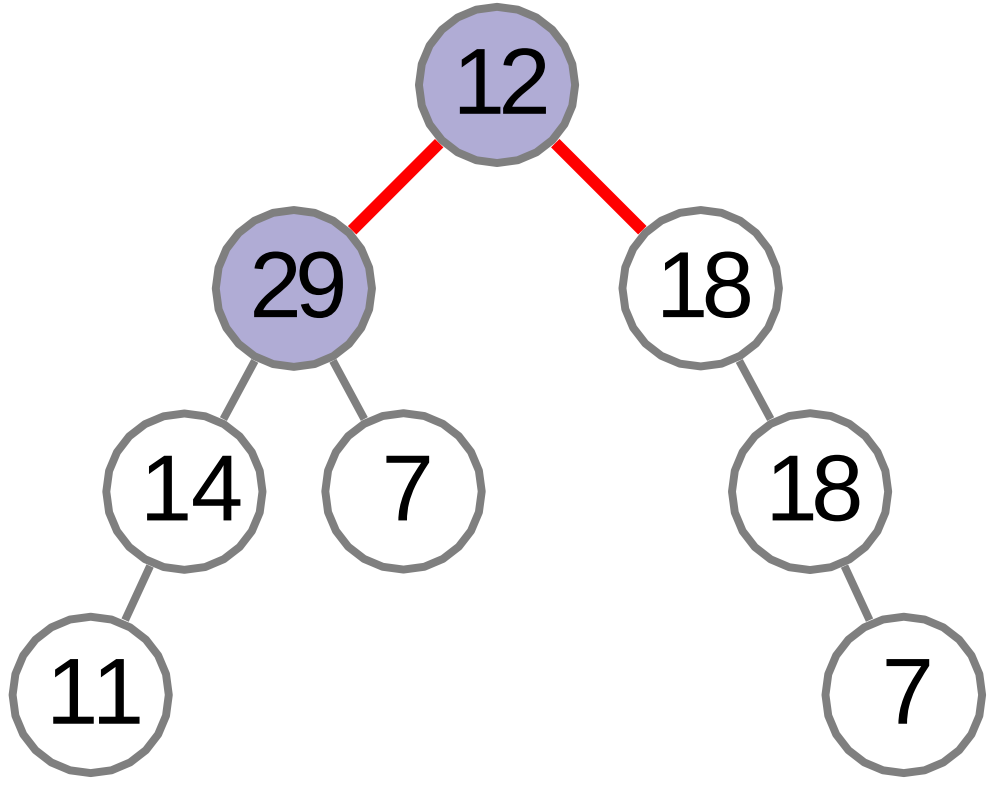


SiftDown

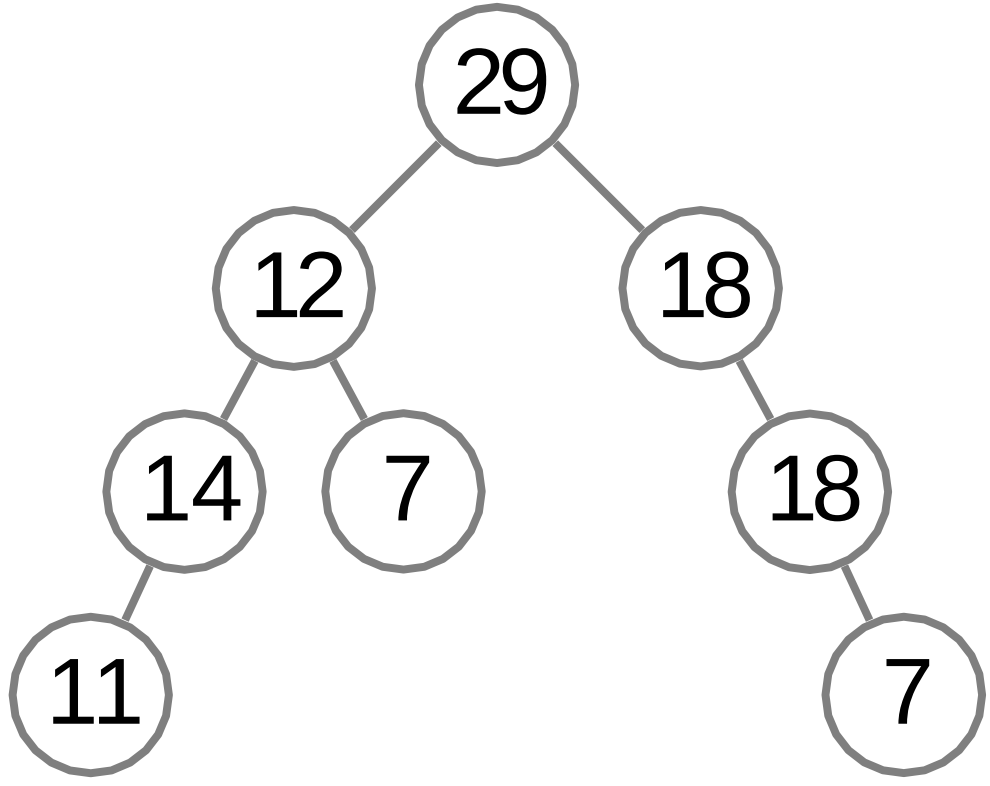
for this, we swap the problematic node with larger child until the heap property is satisfied



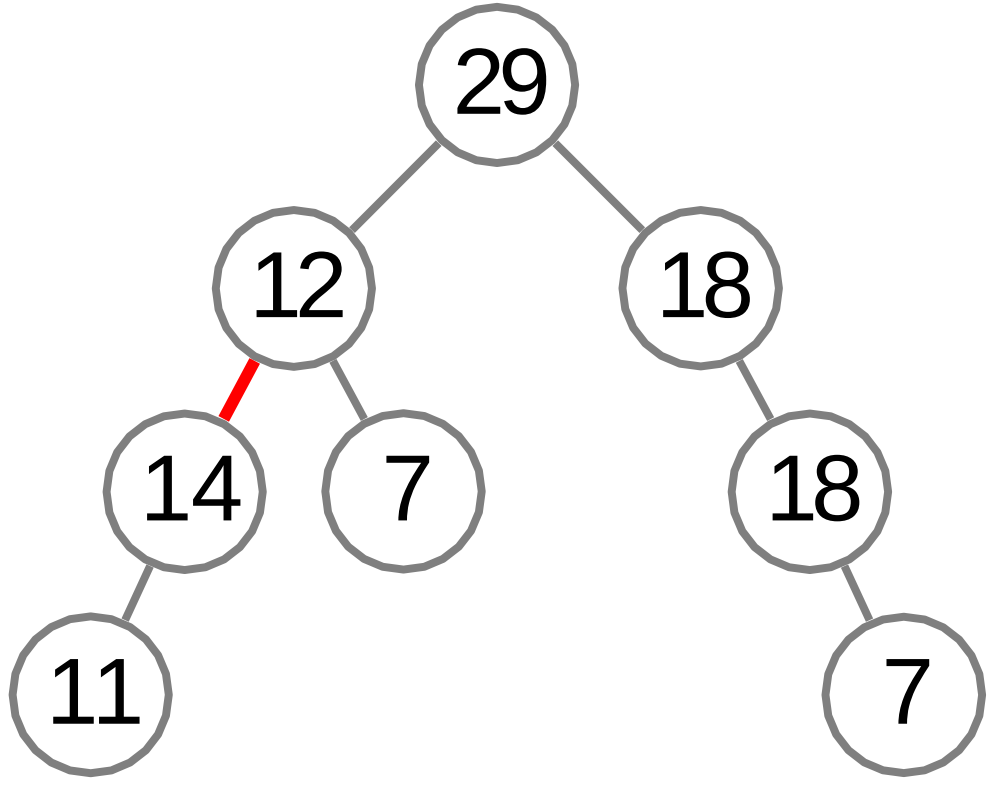
SiftDown



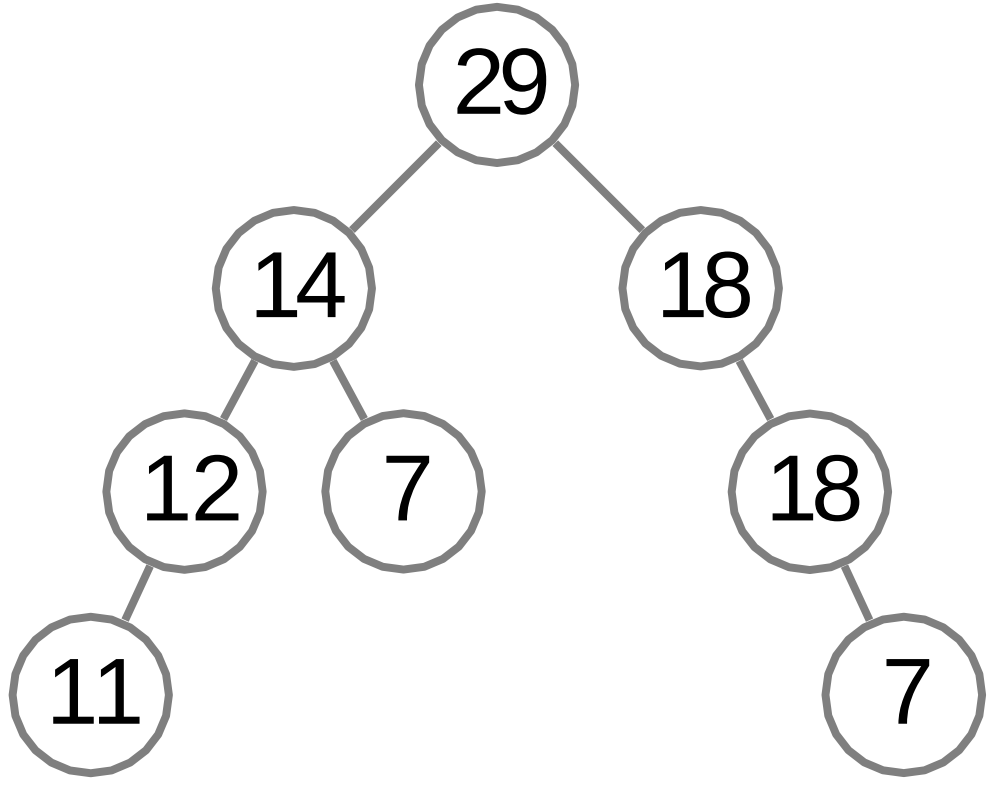
SiftDown



SiftDown

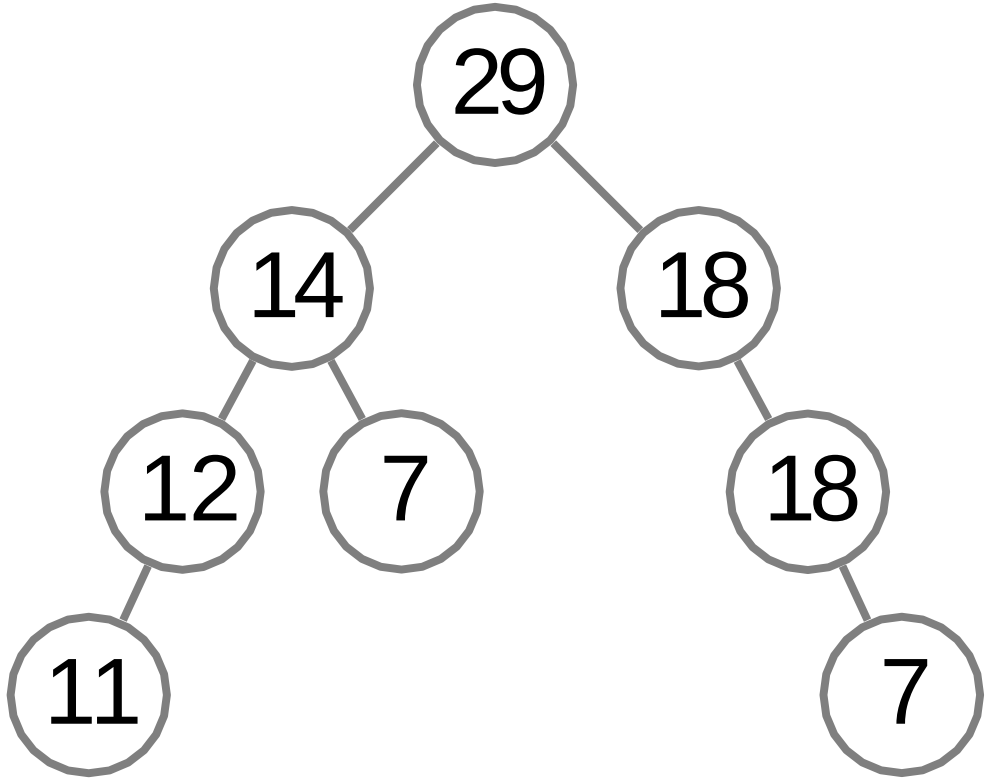


SiftDown

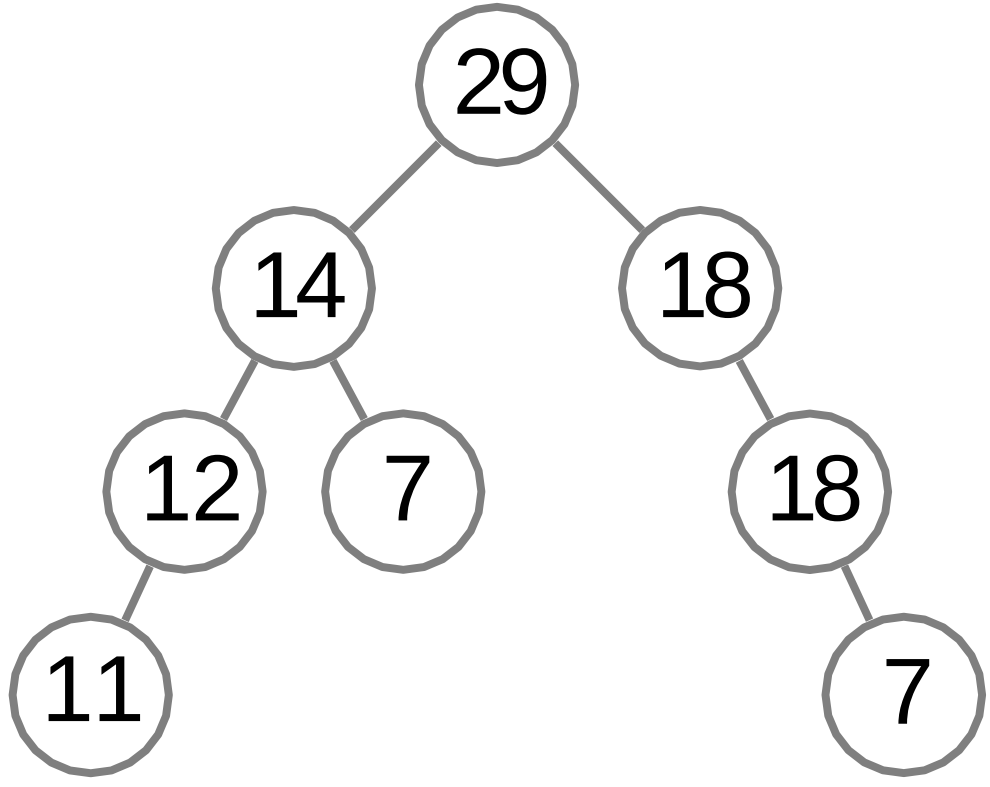


SiftDown

we swap with the larger child which automatically fixes one of the two bad edges

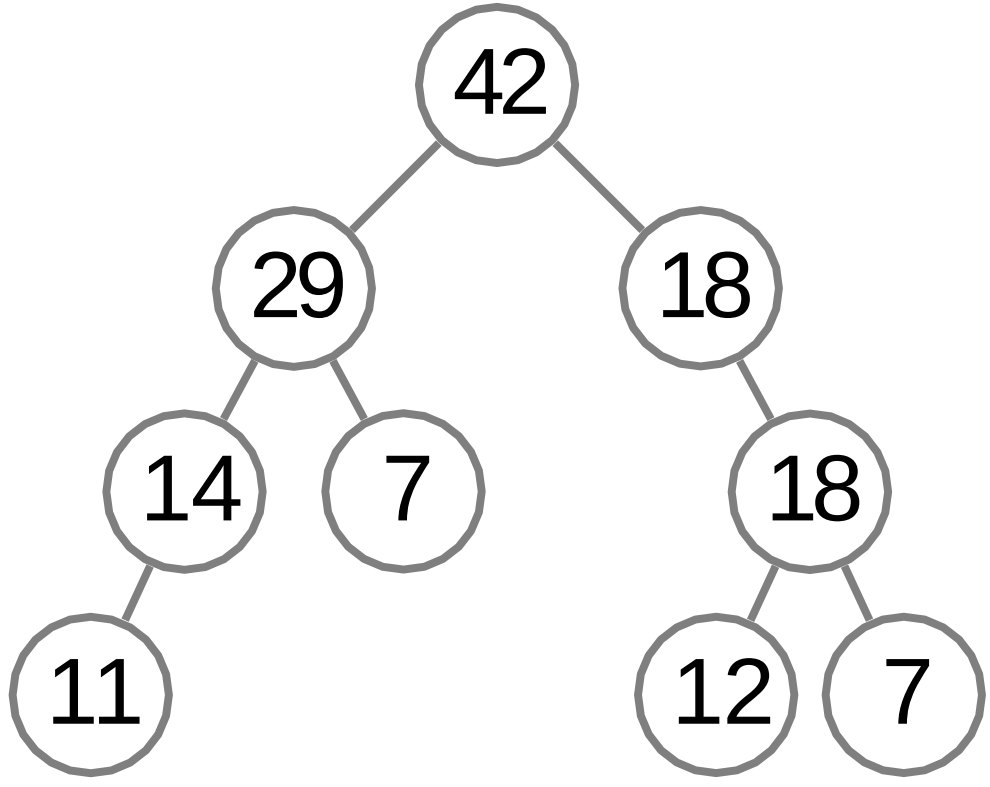


SiftDown



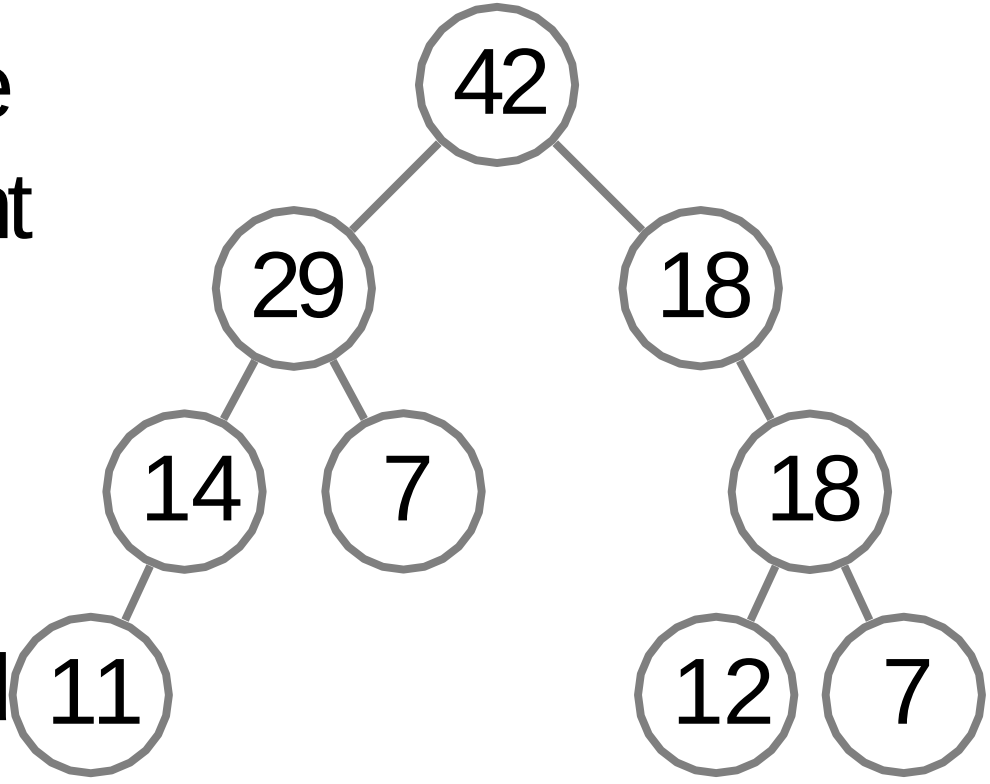
running time: *O*(tree height)

ChangePriority



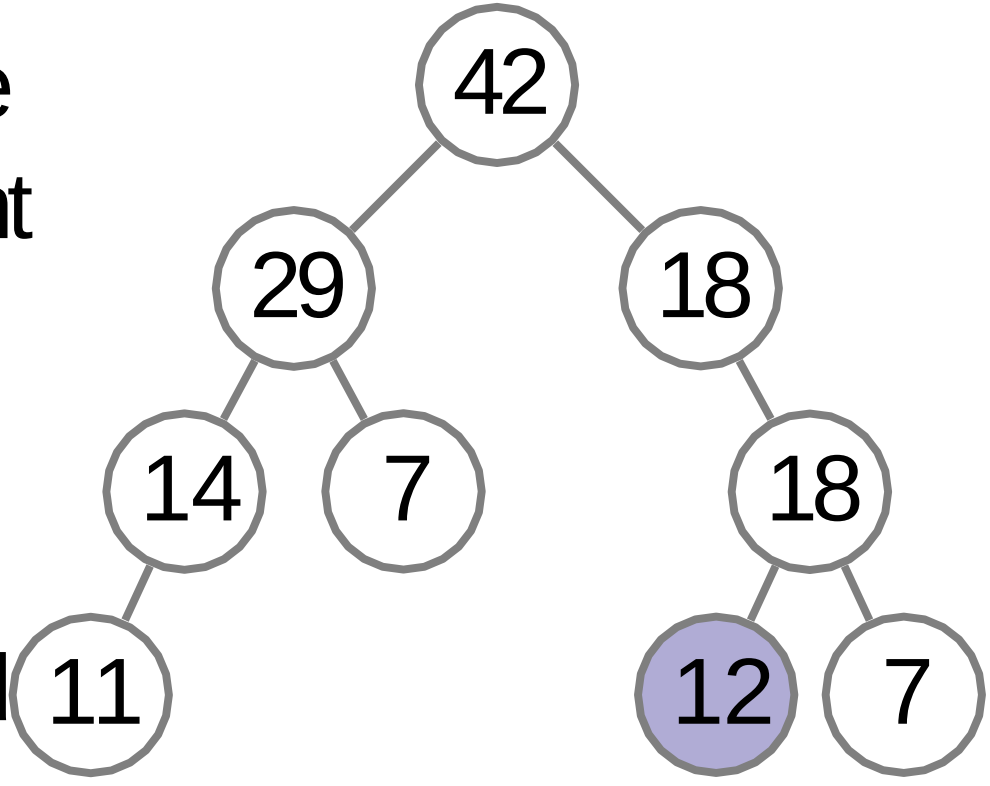
ChangePriority

change the pri-
ority and let the
changed element
sift up or down
depending on
whether its pri-
ority decreased
or increased



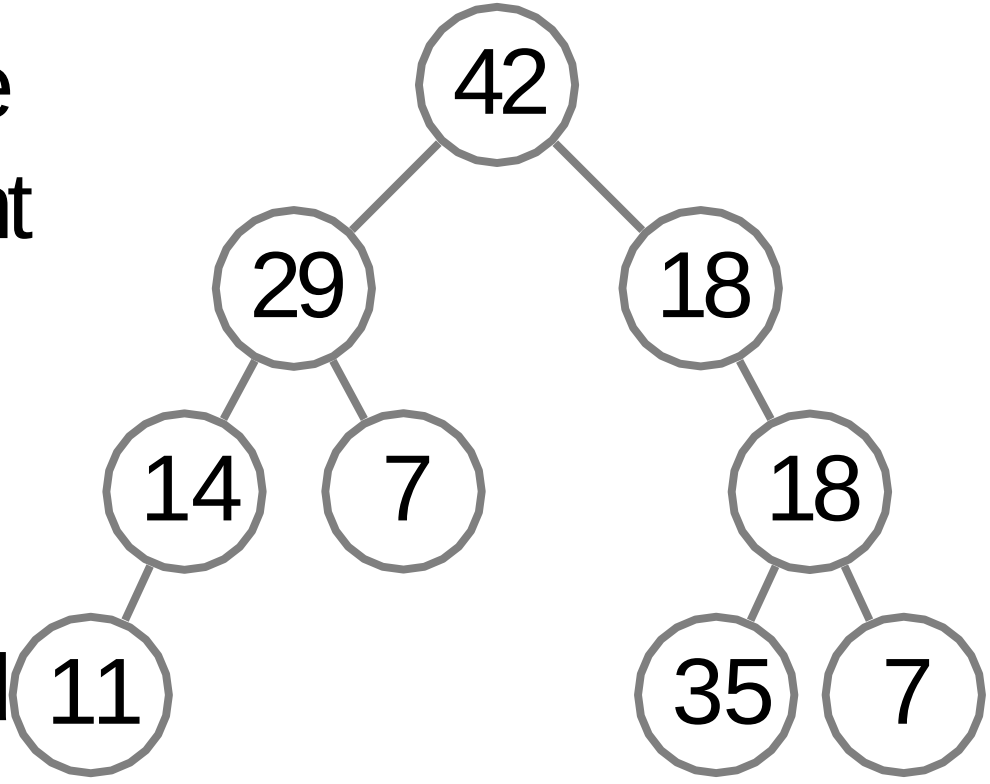
ChangePriority

change the priority and let the changed element sift up or down depending on whether its priority decreased or increased

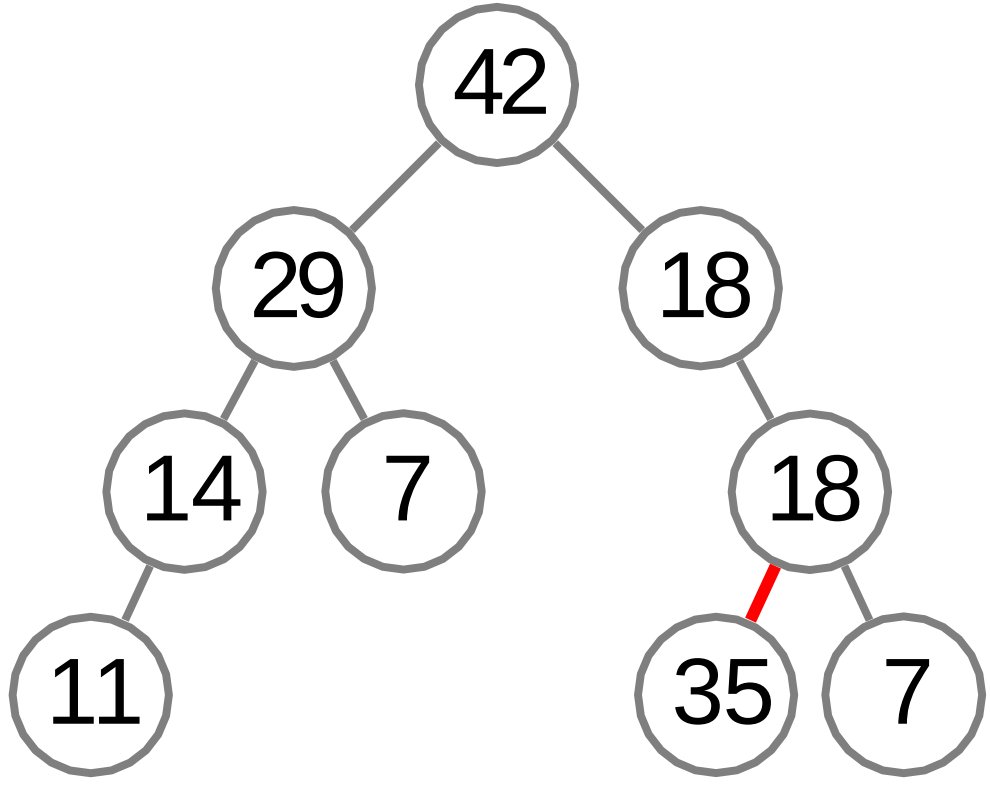


ChangePriority

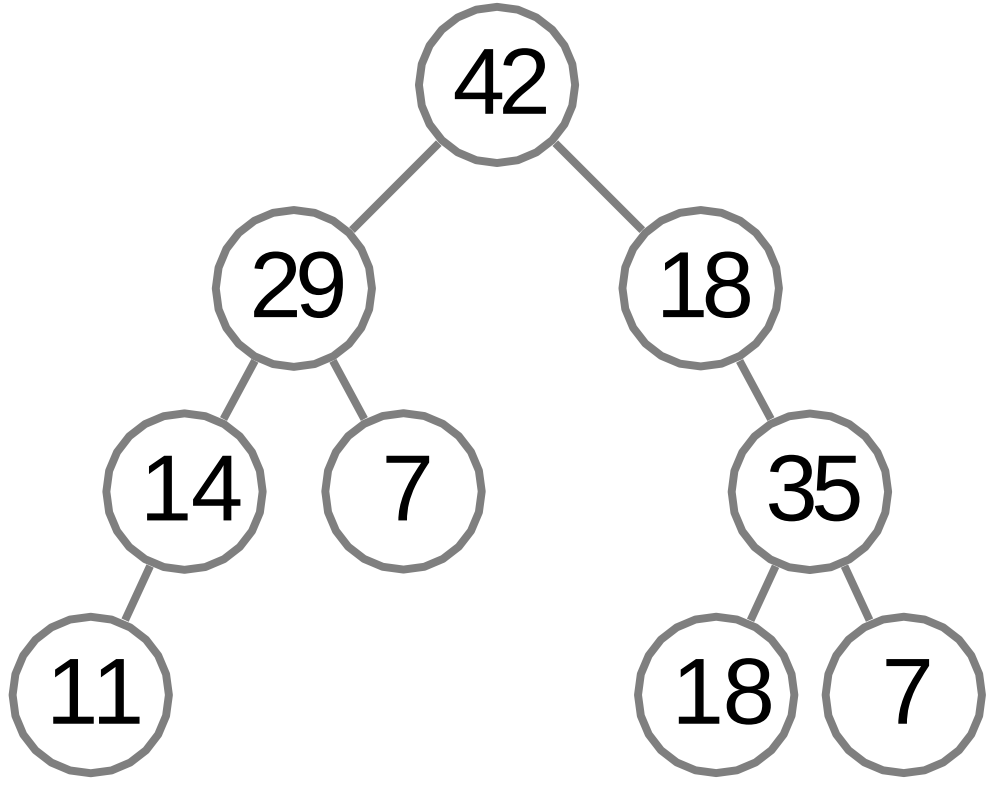
change the pri-
ority and let the
changed element
sift up or down
depending on
whether its pri-
ority decreased
or increased



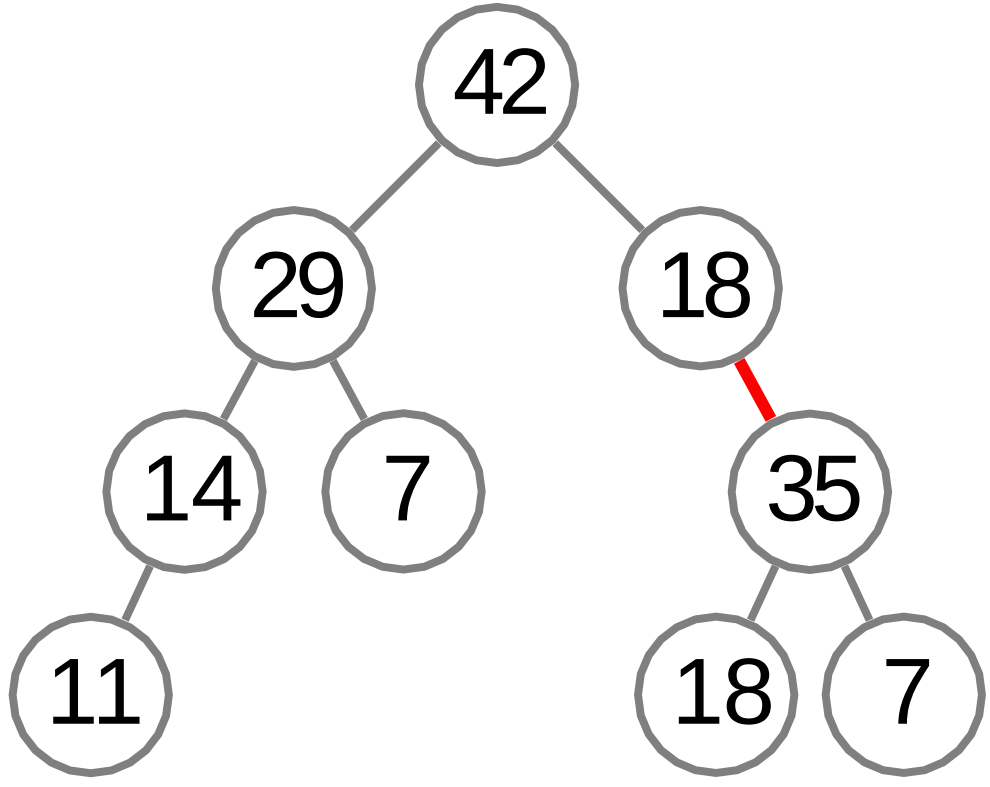
ChangePriority



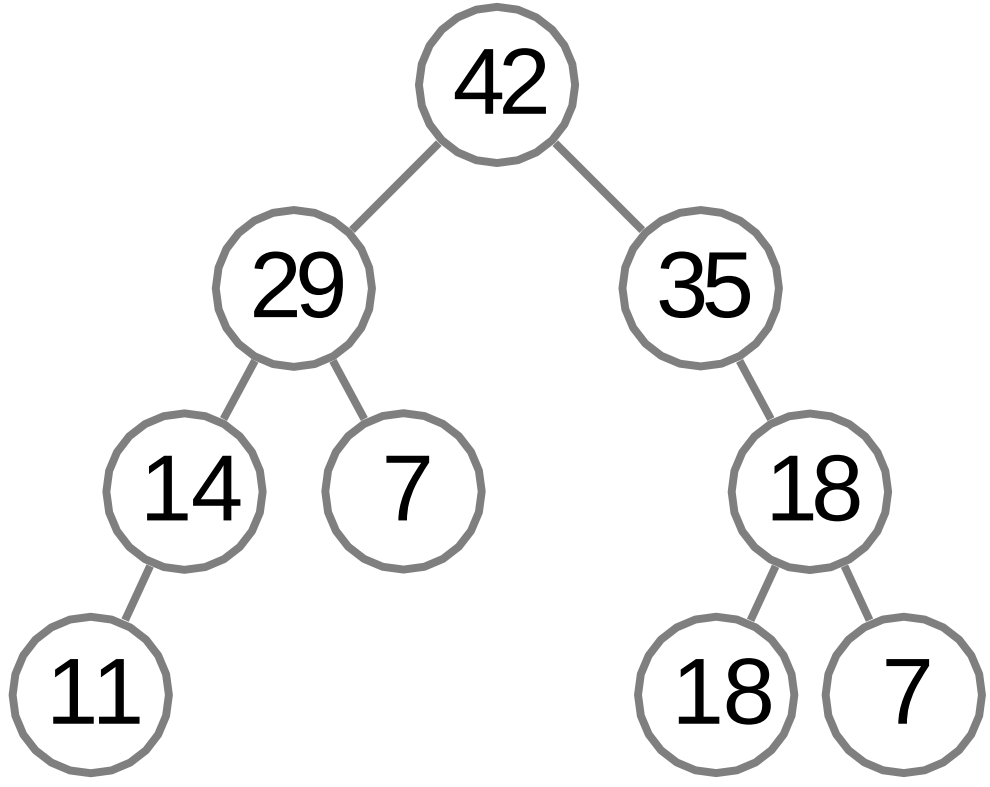
ChangePriority



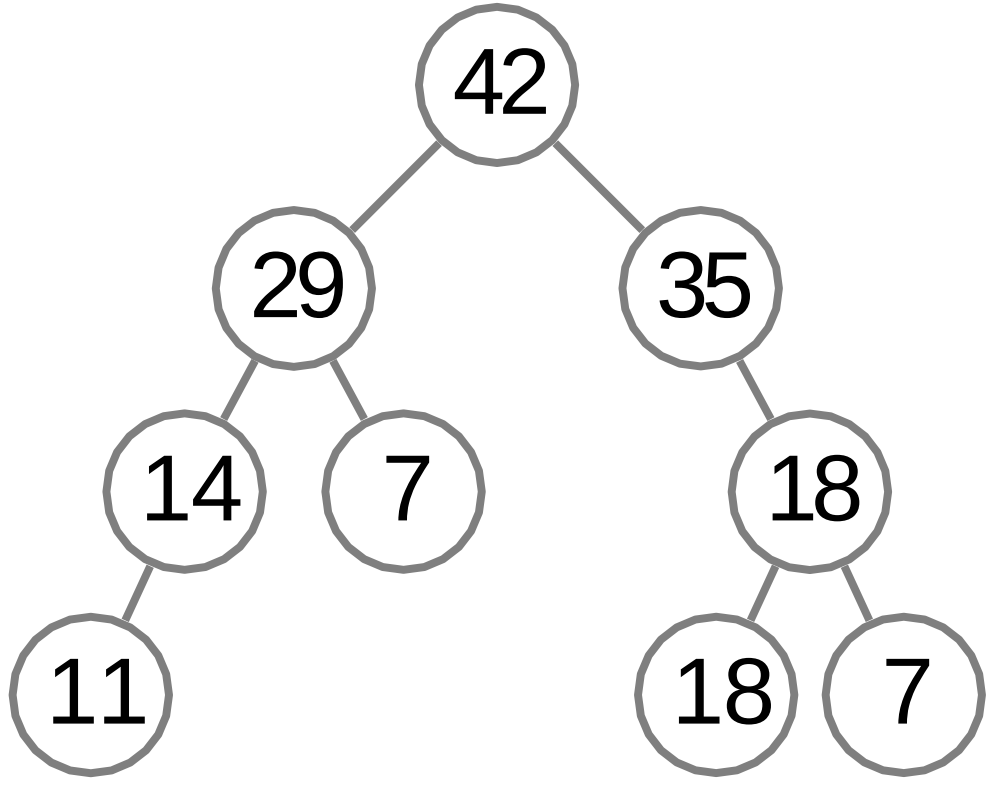
ChangePriority



ChangePriority

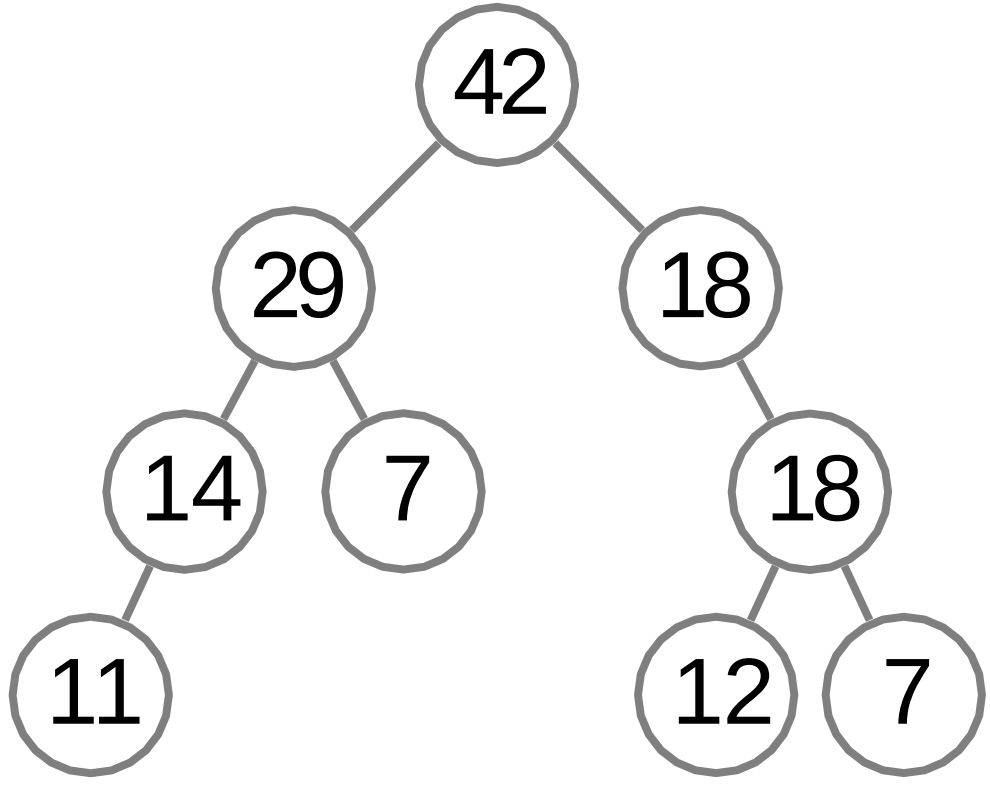


ChangePriority



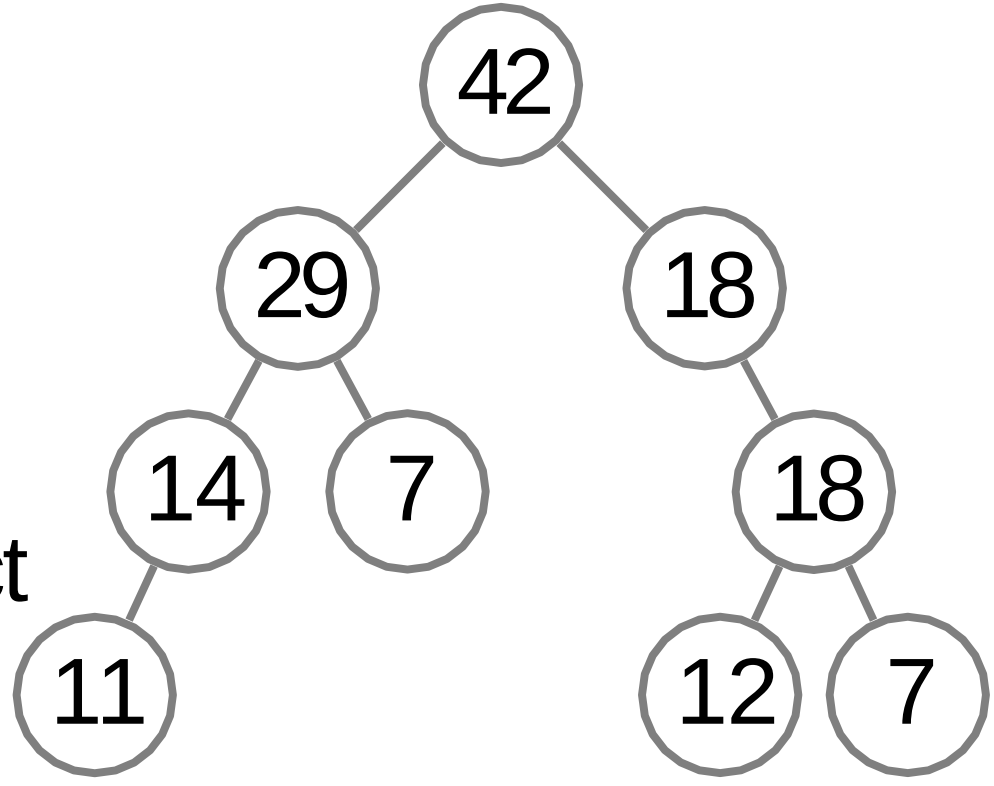
running time: O(tree height)

Remove

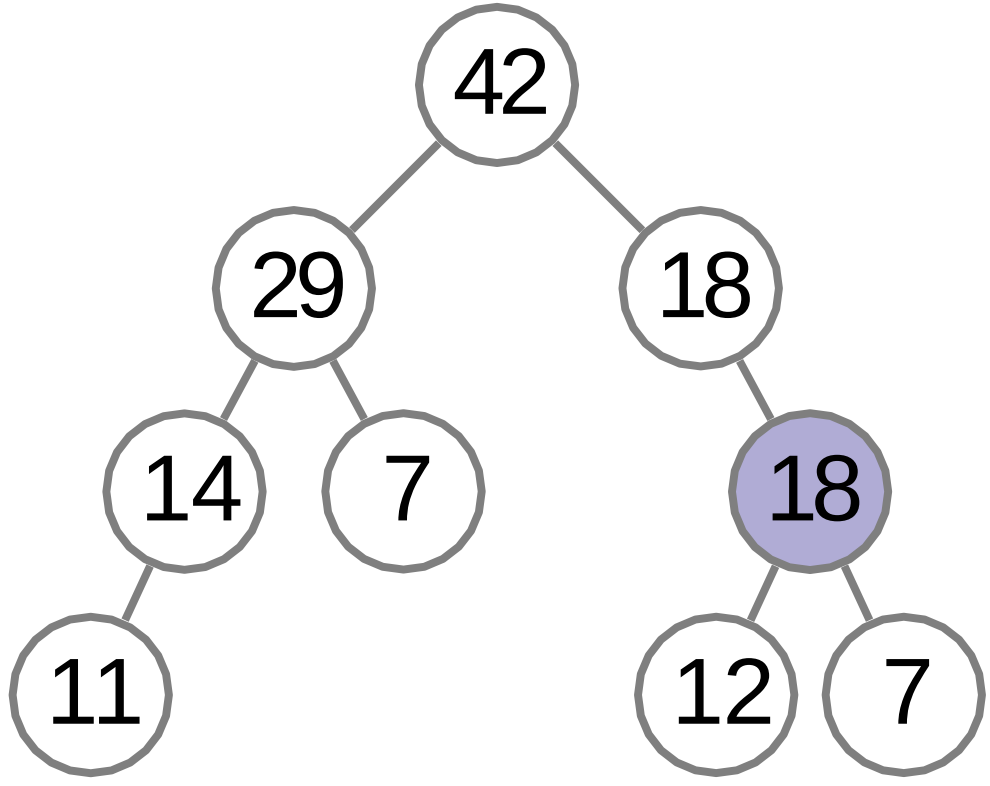


Remove

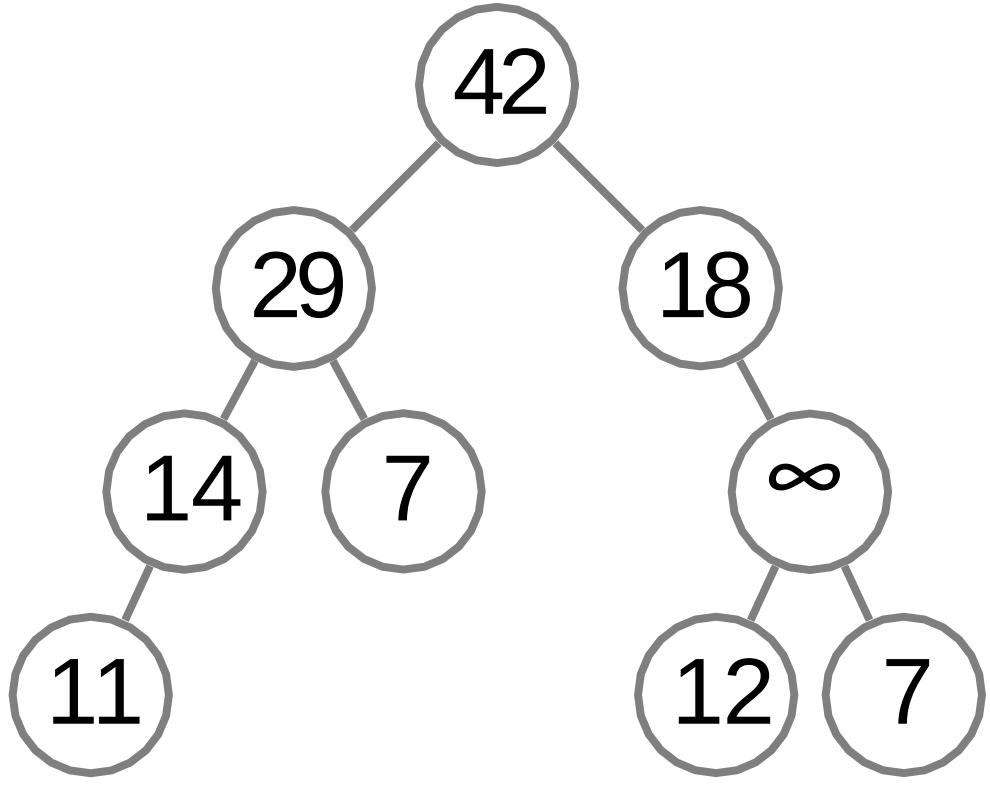
change the priority of the element to be, it sift up, and then extract maximum



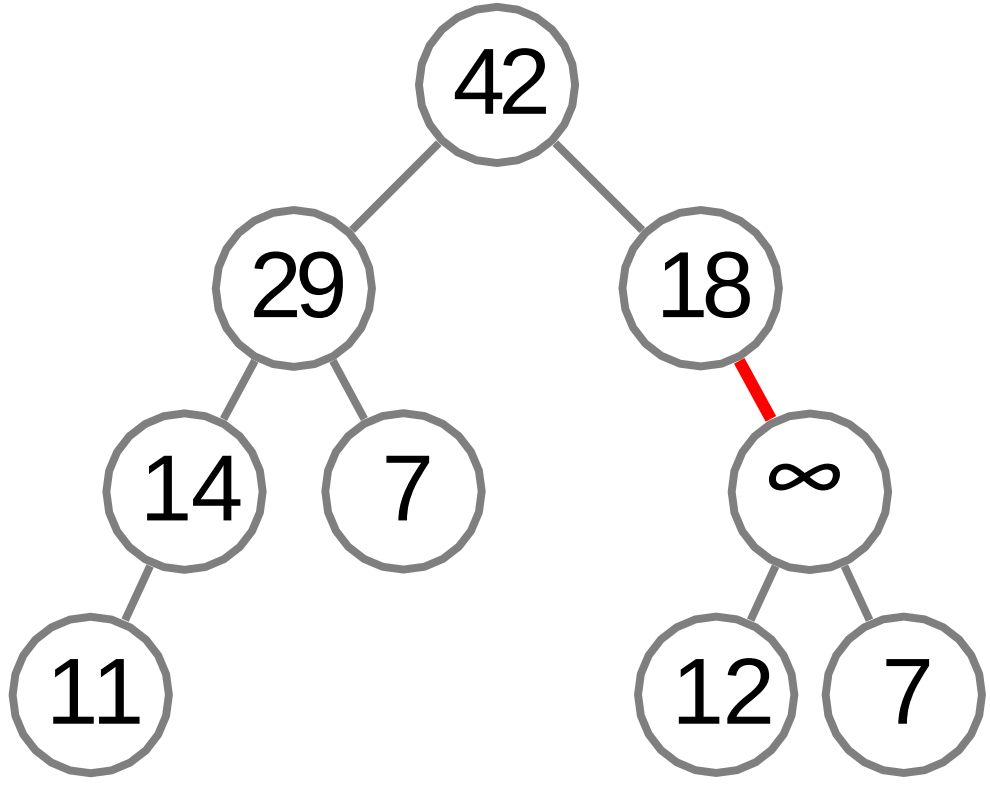
Remove



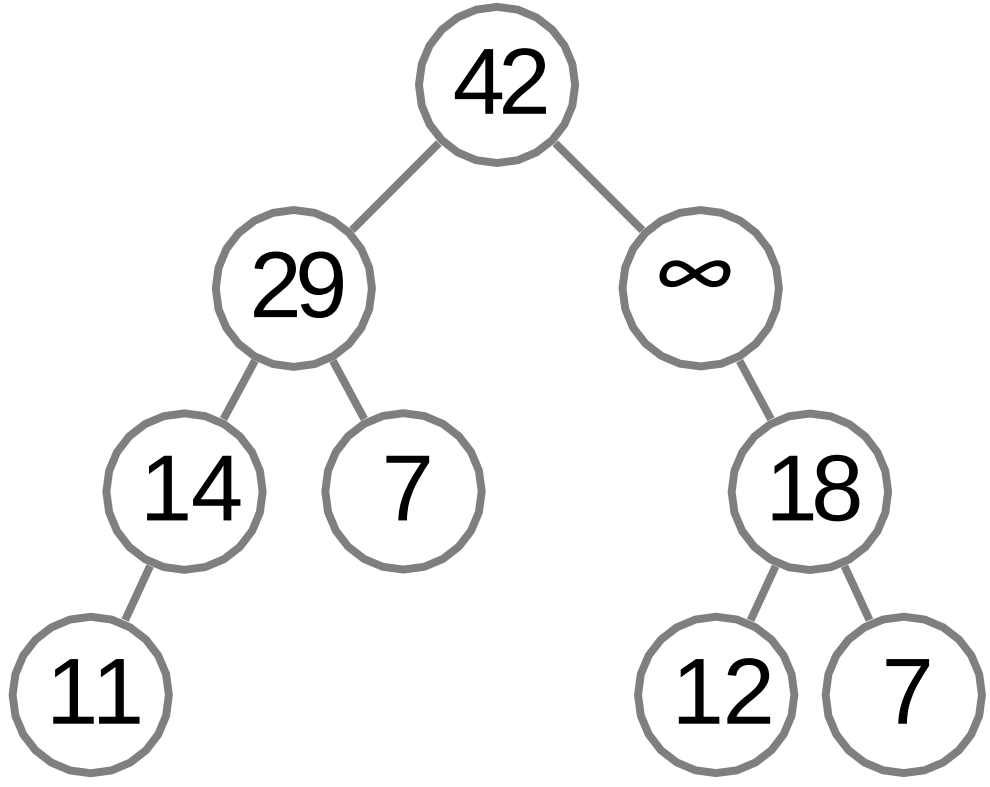
Remove



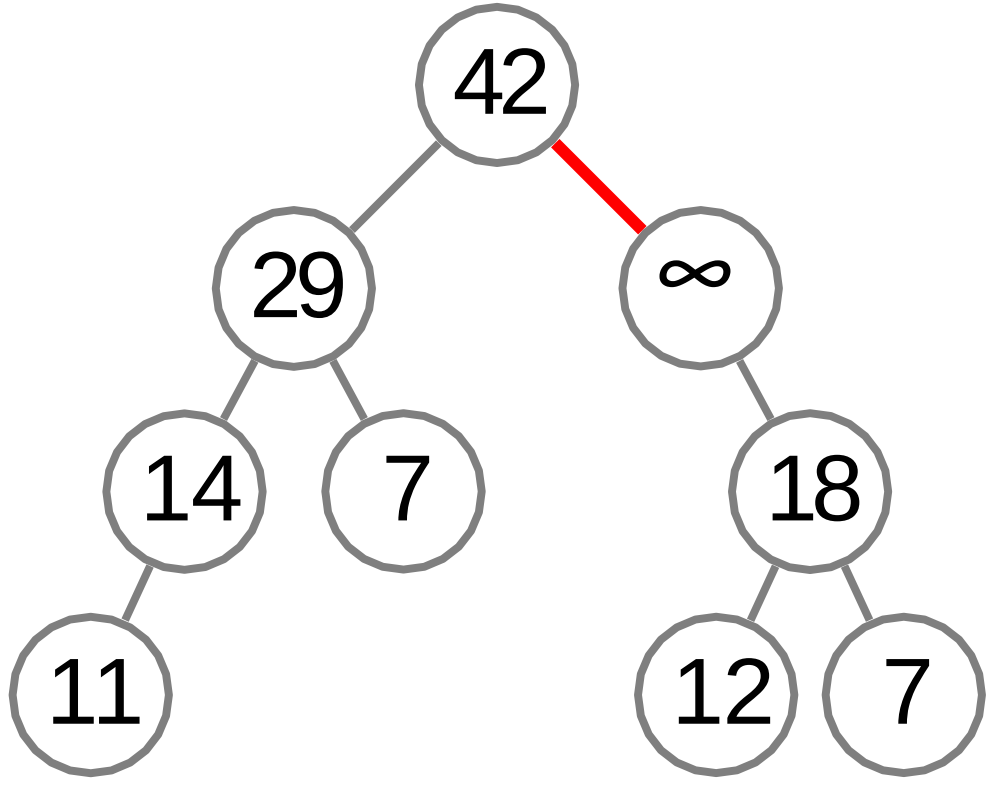
Remove



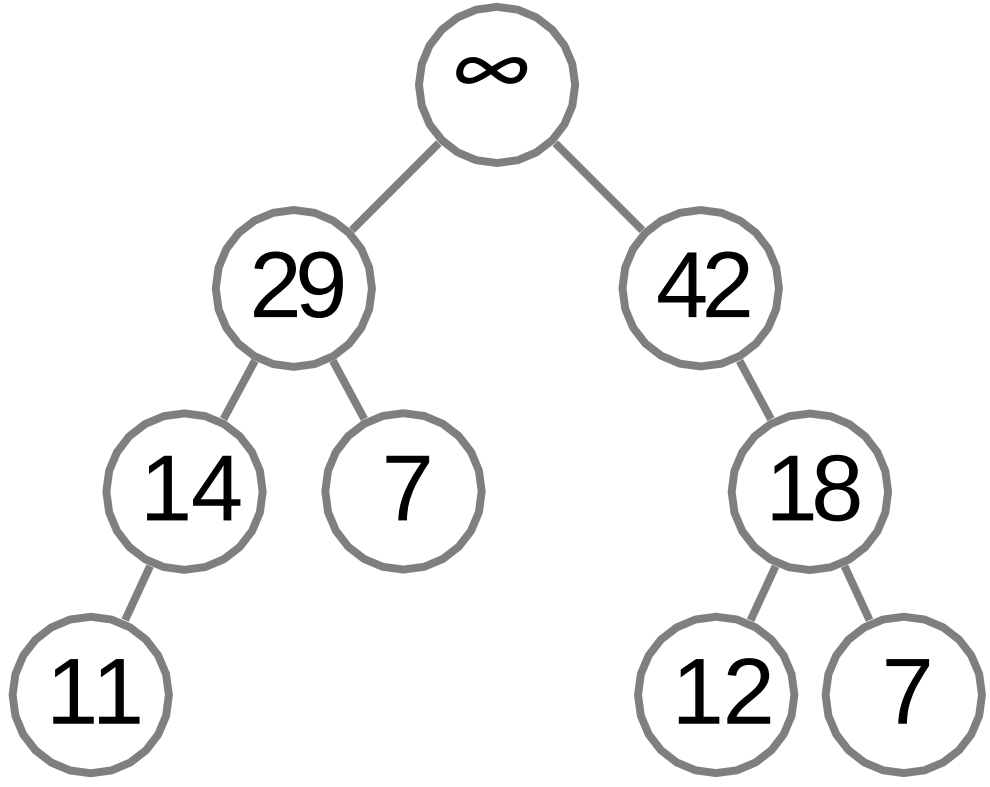
Remove



Remove



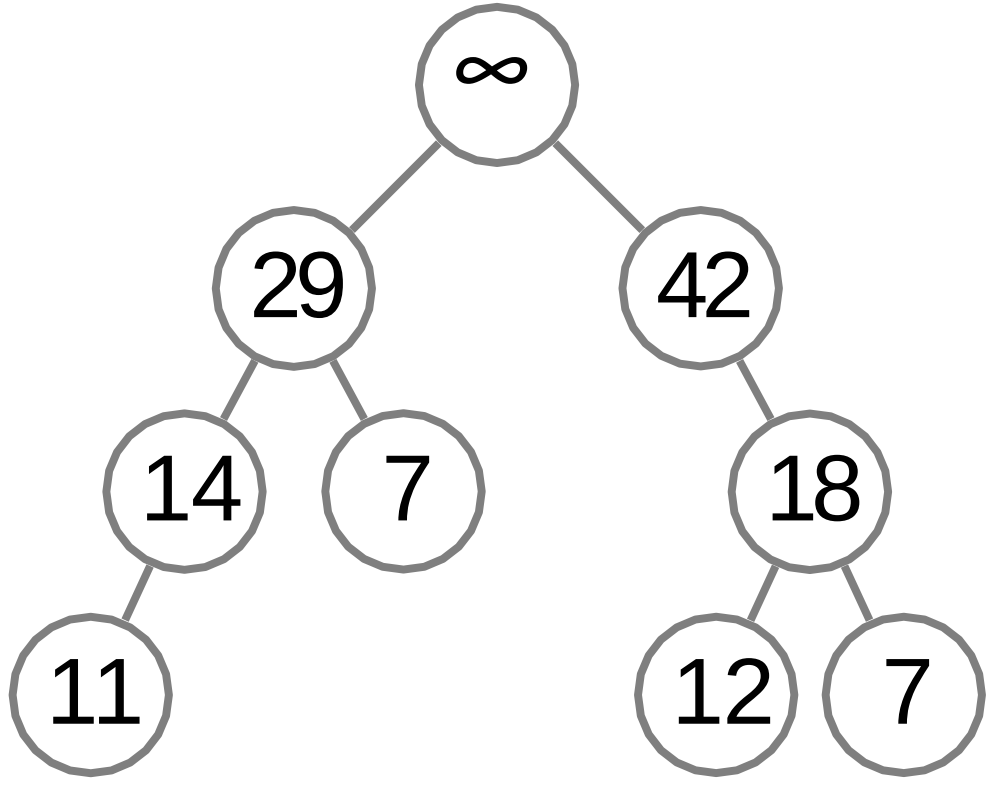
Remove



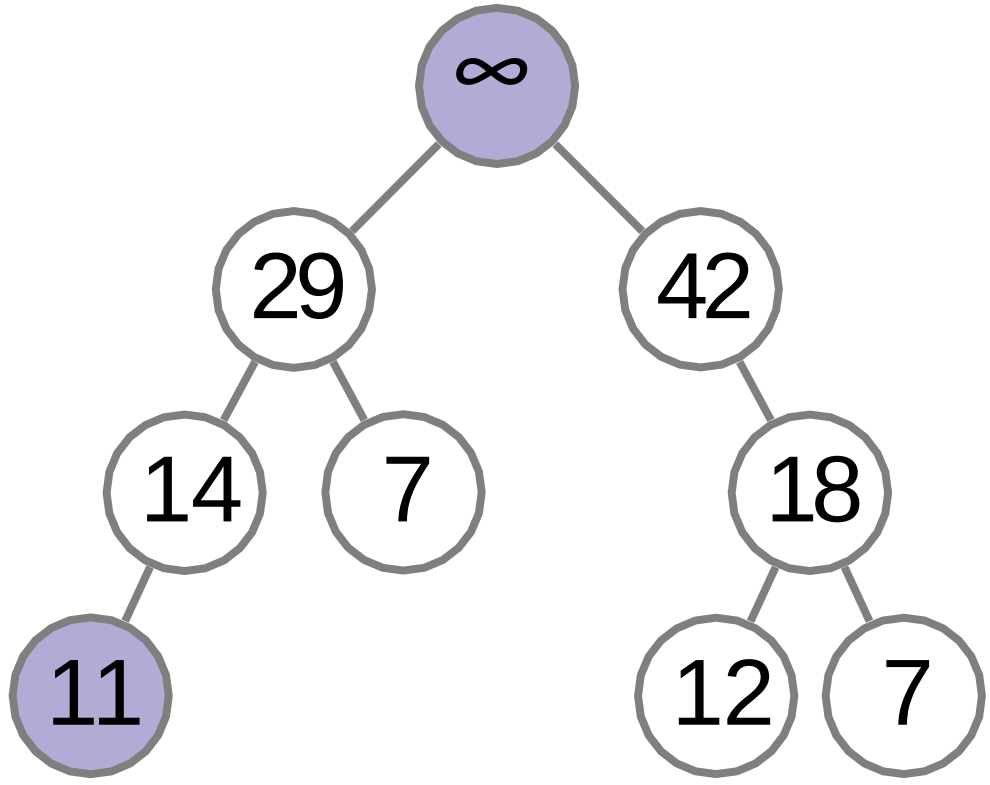
Remove

now, call

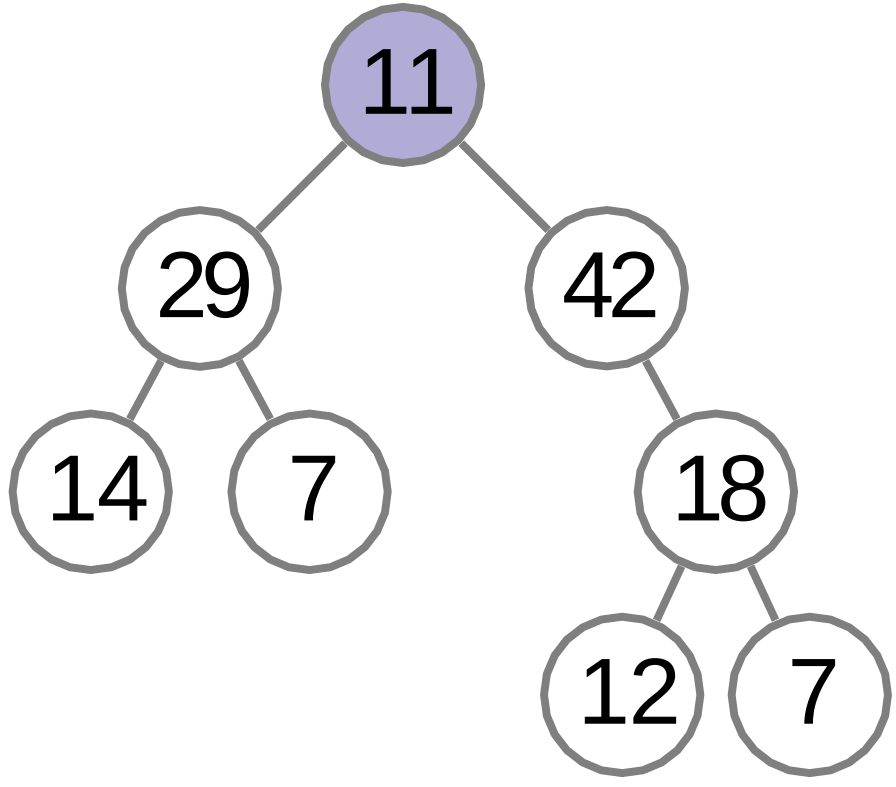
`ExtractMax()`



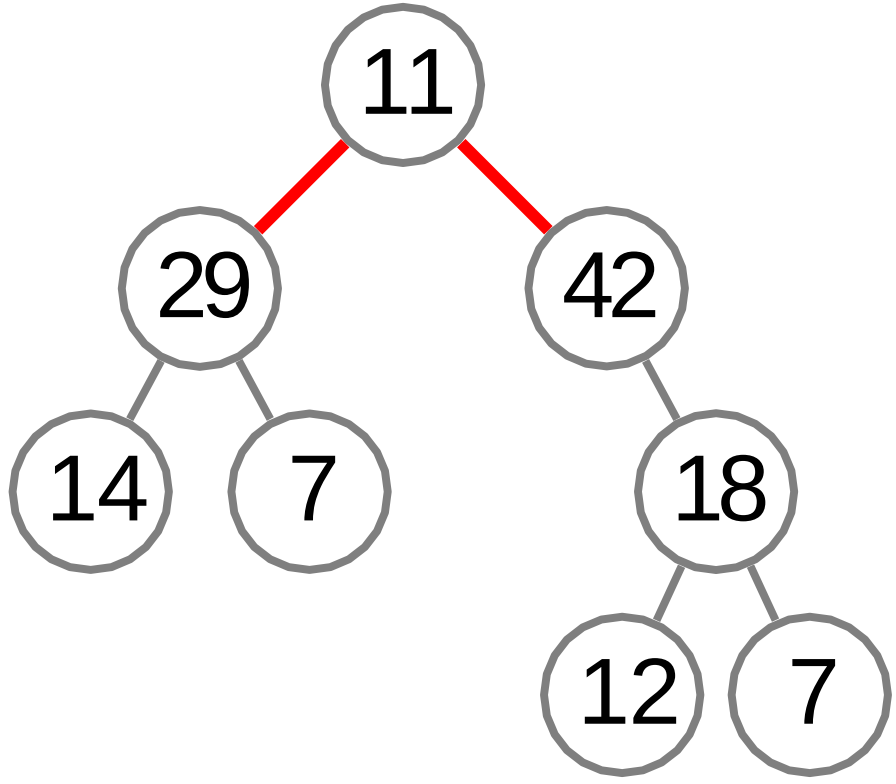
Remove



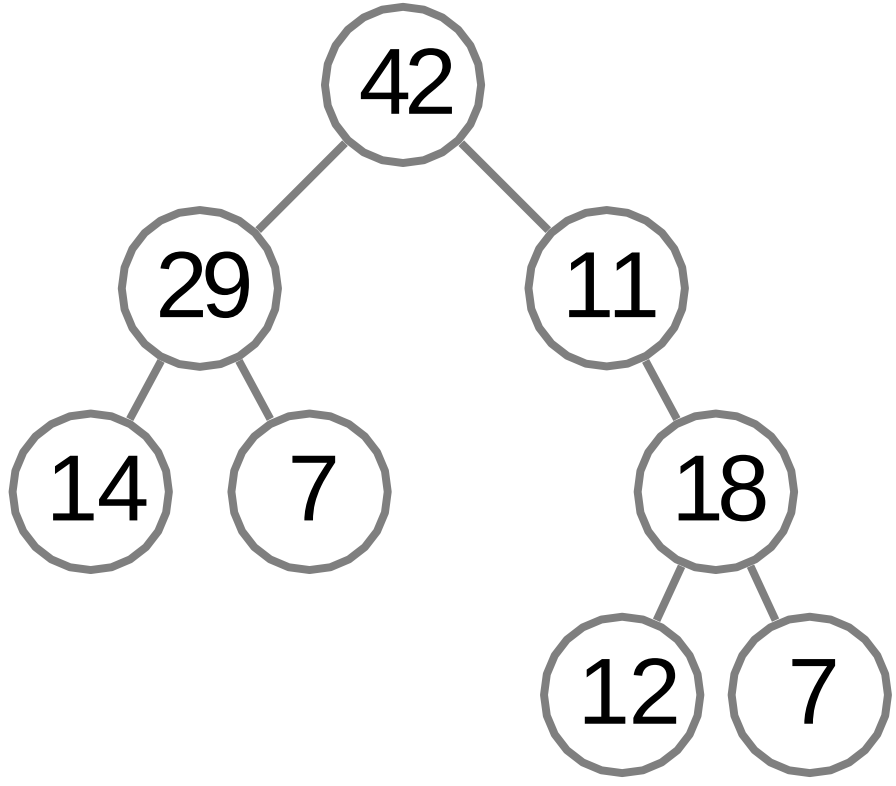
Remove



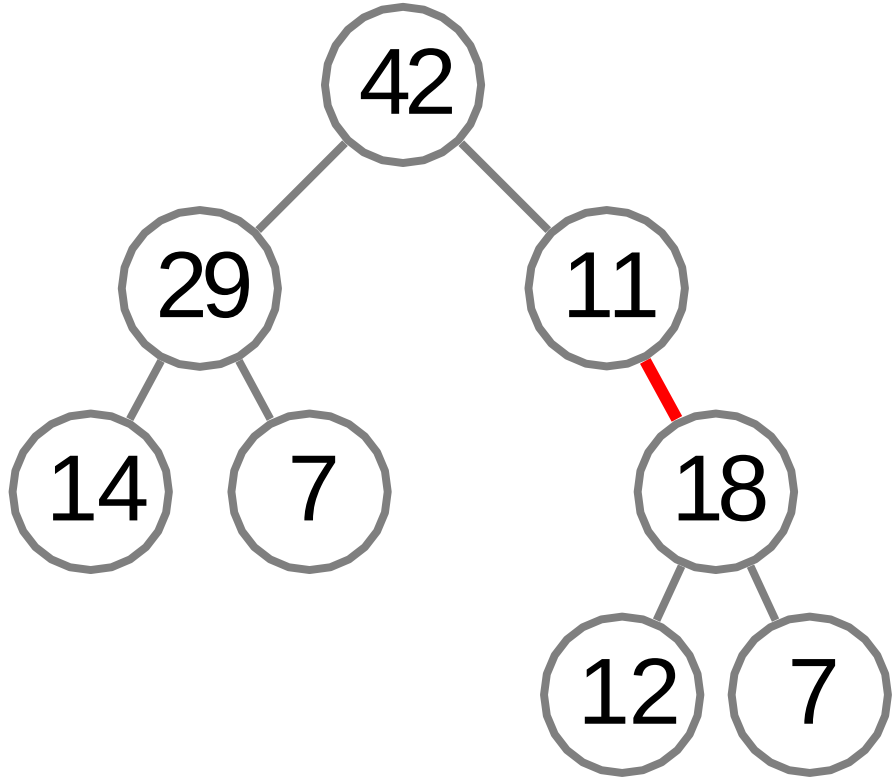
Remove



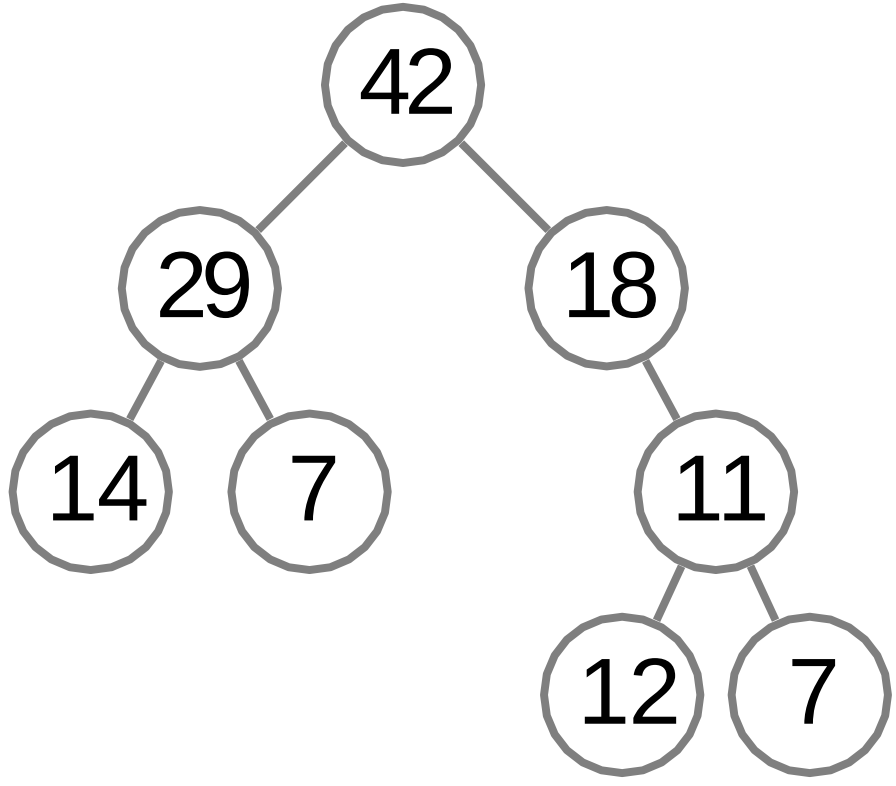
Remove



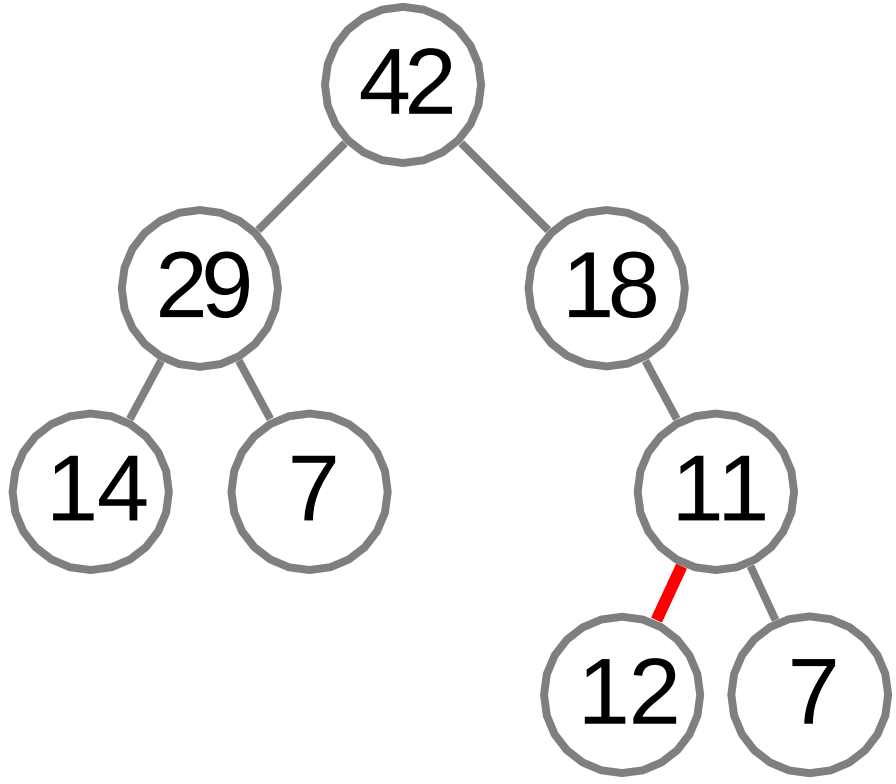
Remove



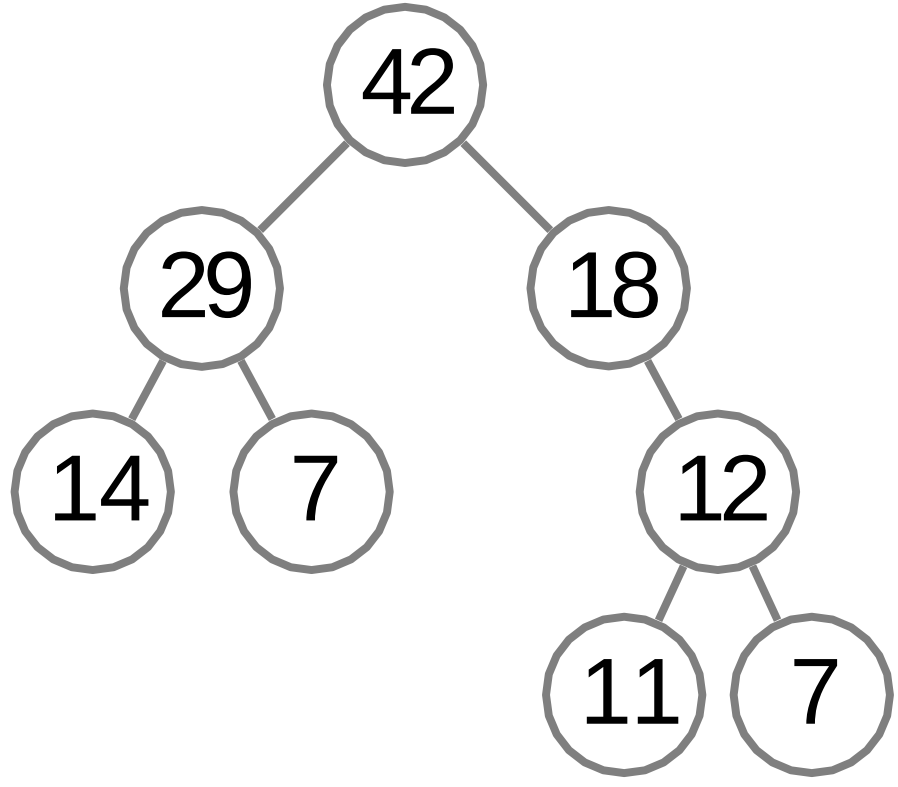
Remove



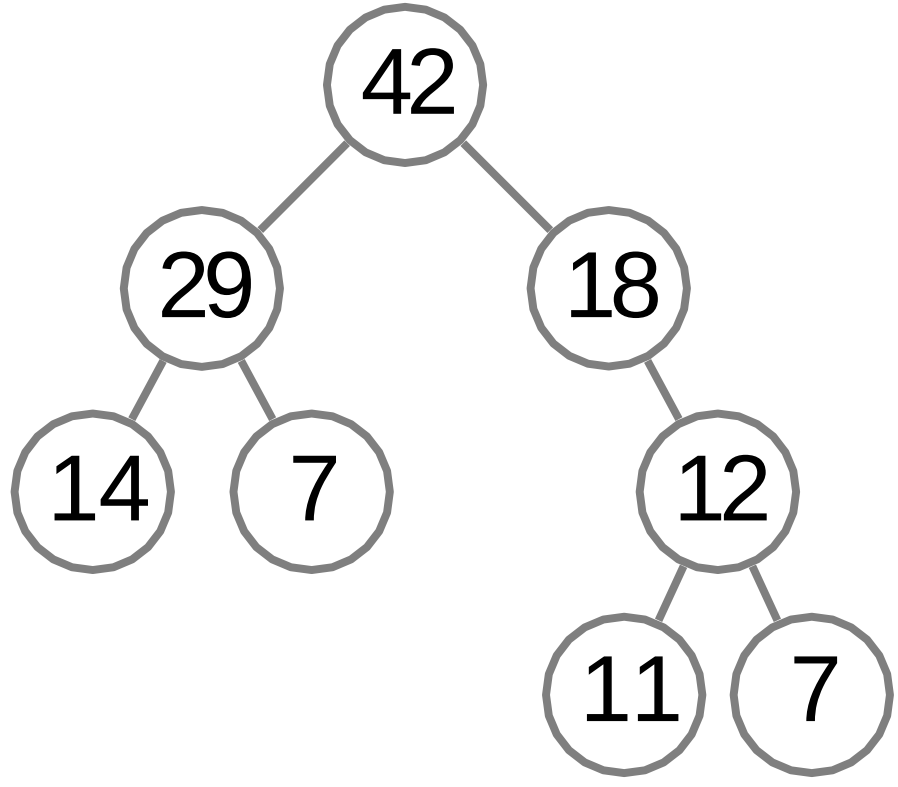
Remove



Remove



Remove



running time: *O*(tree height)

Summary

- `GetMax` works in time $O(1)$, all other operations work in time O(tree height)

Summary

- `GetMax` works in time $O(1)$, all other operations work in time O(tree height)
- we definitely want a tree to be shallow